

Learning Journey Assistant: System Maintenance Document

Learning Journey Assistant — System Maintenance Document

How to read this document

1. System overview
2. User stories: the thread
3. Design and implementation
4. Deployment and environment
5. Ongoing maintenance
6. Testing
7. Usability trial results
8. Future work

Appendix A. Glossary

Appendix B. Metrics reference

Appendix C. Source documents

Appendix D. Change log

Learning Journey Assistant — System Maintenance Document

Status	DRAFT 0.6 — for team review. Sections marked $\triangle$ <i>TO FILL</i> depend on the 4/5 October review, which has not happened yet.
Date	2 October 2026
Describes	main at cd98ce8 (2 Oct 2026, after PR #49) plus the open code pull requests #25 (clusters page), #52 (practice quiz) and the IOLG-106 trajectory branch this revision ships on, labelled where they matter. PR #48 holds the previous revision of this document.
Project	CSE5IDP Industry Development Project, Semester 2 2026, La Trobe University, Group 3 (Jira project IOLG)
Project owner	Dr Scott Mann
Team	Allan Campton (architecture, data model, gap engine), Ayesha Mosaddeque (CI, security scanning, Moodle extraction), Istiaque Bhuiyan (LLM layer, grounding, generation), Anup Tumbalam Gooty (QA, acceptance verification), Sui Lung Tang (risk, dashboard views)
Repository	https://github.com/yetanotherpassword/LearningJourneyAssistant
Companion documents	Repository README.md (quick start), the User Document (docs/handover/user-document.md: how the numbers are made, with worked examples, and the dashboard walkthrough), the two ADRs under docs/adr/, the tender (Tender Document.docx), and the Sprint 5 evidence set under docs/ (Appendix C lists each file)

How to read this document

This document is written for whoever inherits the Learning Journey Assistant (LJA): the project owner, or the team that picks the system up next semester. It assumes you can read Python and run a terminal, but nothing about this codebase.

The user stories in §2 are the thread that runs through the whole document. Every story carries the same identifier in the design (§3), the deployment steps (§4), the test table (§6) and the usability trial (§7), so you can pick any story and follow it from requirement to code to evidence.

The diagrams in §3 are images under `docs/handover/diagrams/`, rendered from the Mermaid source files beside them. To change a diagram, edit the `.mmd` file and re-render with `npx -y @mermaid-js/mermaid-cli -i diagrams/NAME.mmd -o diagrams/NAME.png --size 1800 -s 2`.

If you only have ten minutes: read §1, then §4.3 (clean-clone install), then §5.1 (the repository map). If the system is broken, go to §5.6 (troubleshooting).

Every figure the system calculates or displays is defined in Appendix B, with its formula, where it is calculated and shown, and the status of any threshold it uses.

1. System overview

1.1 What LJA does

LJA reads students' assessment results, works out which subject intended learning outcomes (SILOs) across *different* subjects describe the same underlying competency, and then measures each student against each competency **relative to that student's own profile**, not against a fixed pass mark. Where a student is weak in a competency across more than one subject, LJA calls that a *persistent gap*. A read-only dashboard shows the cohort and each student; a separate command generates a learning plan for one student that is validated, name by name, against the data it was built from.

The problem it addresses, from the tender: students "lack a reliable way to understand what they have mastered, where gaps persist, why those gaps matter for later subjects, and what they should study next", and teaching staff "have no aggregated view of where a cohort is weak against SILOs".

1.2 What exists today

Capability	State on <code>main</code> (2 Oct 2026)
Load results from the project owner's Excel workbook	Working
Load results from a Moodle 5.2 database (rubric fills, read-only role)	Working for one seeded subject (CSE110I); read-only enforcement observed on the dev instance (IOLG-111, §6.5)
Semantic clustering of SILOs into cross-subject competencies via an LLM, with coverage validation and retry	Working; staff review of the five reference-run clusters recorded (IOLG-124, §6.5)
Staff confirmation gate on the clustering (pending / confirmed / rejected)	Working (CLI)
Relative gap detection (median/MAD within each student's profile, with absolute guards)	Working; thresholds still unratified, but a sensitivity curve now exists (§3.9)
Dashboard: cohort list, statistics, cohort drill-down, per-student gaps with evidence, unreviewed-AI banner	Working
Dashboard: strengths view and student picker	Working (IOLG-112, PR #23)
Dashboard: progress across subjects; recommended next actions from the learning plan	Working (IOLG-107, PR #33; IOLG-122, PR #35)
Trajectory: declared subject sequence, subject chain on every gap card, configurable stable band	Working (IOLG-106, PR #53)
Generate or regenerate a plan or quiz from the student page, opt-in, with the command's own progress	Built on the IOLG-137 branch (D20)
Dashboard: priority groups, <code>/run</code> provenance page, <code>/glossary</code> , every count tile links to its list, chart enlarge, bounded scroll boxes	Working (IOLG-134, PR #43)
Dashboard: outcome quality (<code>/silos</code>), competencies, subjects and assessments pages, competency traceability diagram	Working (IOLG-132, PR #28; PR #43)
Dashboard: competency clusters page linked from the banner	Open PR #25
Grounded learning plan for one student (fails closed on any invented name)	Working; ten-plan audit passed the machine checks 10/10, human review pending (IOLG-121)
Study-strategy recommendations per gap	Working (IOLG-123, PR #26)
Practice quiz per gap (multiple choice or written task, blind educator review)	Open PR #52 (IOLG-136), a thin slice; see §3.10
Longitudinal / A-B research export with pseudonymised ids	Working (IOLG-120, PR #32)
Synthetic cohort generator from a YAML subject catalogue, with ground truth	Working (IOLG-113, PR #24)
Handbook crawl of real La Trobe SILOs and the embedding competency tagger	Working, code only (PR #29); crawled data stays out of git pending the owner
Committed reference run so the pipeline and dashboard work offline	Working (PR #22)
CI: lint, tests, secret scan, dependency audit; branch protection on <code>main</code>	Working; 261 tests pass
Sprint 5 evidence: validation worksheet, grounding audit, security evidence, compliance checklist, risk register, UAT checklist	Recorded under <code>docs/</code> (§6.5)
Clean-machine rebuild by a non-author	Not performed; four preflight findings recorded (IOLG-118)

1.3 Guardrails the system is built around

These come from the tender and the project owner and shape most of the design decisions in §3.7:

- AI recommendations never override academic grading; mastery estimates are formative indicators, not official evaluations.
- Every AI output is grounded in structured subject data. The LLM is never allowed to invent a SILO, subject, assessment or competency, and the code enforces this rather than trusting the prompt.
- No real student data is used. Every student in the repository is synthetic (`STU0001...`) or from the anonymised, modelled workbook the owner supplied.
- Local-first. The Moodle instance, the pipeline and (optionally) the LLM run on hardware under the team's control. Cloud migration is deferred, not forbidden.

- Least privilege everywhere: a read-only database role, a four-function web-service token, no admin credentials, secrets never committed.
 - Nothing ever writes to Moodle tables.
-

2. User stories: the thread

The stories below are the ones the team agreed for the Sprint 5 user acceptance test (UAT). S stories exercise the command line, D stories the dashboard. Four engine-level stories (E) are added here because they carry the algorithmic requirements that the S and D stories depend on. Tender requirement numbers (R1–R12) and Jira keys are given so the thread can be followed into the board.

ID	As a...	I want...	so that...	Tender	Jira	Status
E1	subject coordinator	SILOs from different subjects grouped by the competency they actually describe, not by keyword	a gap in first year can be linked to its consequence in third year	R2, R3	IOLG-82, IOLG-109	Done
E2	subject coordinator	to confirm or reject the AI's grouping before it drives any report	no gap report is produced from unreviewed AI output without saying so	R3, R6	IOLG-82, IOLG-115, IOLG-116	Done
E3	student	my weak competencies identified relative to my own profile, with an absolute floor and ceiling	a strong student's weakest area and a weak student's uniform weakness are both surfaced	R4	IOLG-86	Done; thresholds unratified
E4	student	a learning plan whose every named subject, SILO and assessment is one I actually have	the AI cannot hallucinate advice	R6	IOLG-85, IOLG-108	Done
S1	team member	to run the pipeline on the supplied workbook	a gap report is produced from a clean clone	R11	IOLG-78, IOLG-118	Done
S2	team member	to run the pipeline on a live Moodle database	the production data path is proven	R1	IOLG-104, IOLG-105, IOLG-56	Done for one subject
S3	student / staff	to generate one student's learning plan	the student gets grounded next-step advice	R6	IOLG-108, IOLG-119	Done
S4	researcher	to export results as CSV with a manifest	longitudinal and A/B evaluation is possible	R7	IOLG-120	Done
S5	staff	a rejected cluster to change what the dashboard shows	reviewers can see that a review had effect	R3	IOLG-116	Done
S6	maintainer	the test suite to pass	I know the install is healthy	R11	IOLG-80	Done
S7	student	a study strategy per gap, built from techniques with evidence behind them and tied to my own assessments	I know how to study, not just what	R6	IOLG-123	Done (added in 0.4; not in the UAT checklist)
S8	student	a practice quiz per gap whose every question names one of my SILOs and one of my assessments, reviewed by staff before I see it	I can check whether a gap is closing	R8	IOLG-136	Open PR #52, thin slice (added in 0.4; not in the UAT checklist)
D1	staff	to see the whole cohort with gap counts and sortable columns	I can find who needs attention	R5	IOLG-83	Done
D2	staff	to drill into a cohort and be told what put each student there	every figure is traceable	R5	IOLG-83	Done

ID	As a...	I want...	so that...	Tender	Jira	Status
D3	student	to see my understanding per competency as a chart	I know where I stand	R5	IOLG-83	Done
D4	student	to see my strengths with their basis	I know what to build on	R5	IOLG-112	Done
D5	student	to see each gap with per-subject evidence	the gap is explained, not just labelled	R5	IOLG-83	Done
D6	student	to see progress across subjects	I can see whether a gap is closing	R5	IOLG-107	Done
D7	student	recommended next actions from my learning plan on the dashboard	the plan is where I look	R5, R6	IOLG-122	Done
D8	any user	to switch student from the header	navigation is quick	R5	IOLG-112	Done
D9	any user	to know when the AI grouping is unreviewed, and click through to see which clusters	I do not trust an unreviewed report	R6	IOLG-116, IOLG-131	Done; click-through in PR #25 branch
D10	staff	to name the source record behind any number	the dashboard is auditable	R5	IOLG-83	Done
D11	staff	flagged students split into priority groups by rules the pipeline already applies, with the run's provenance and a glossary on the dashboard	a tile never says a number without saying why	R5	IOLG-134	Done (added in 0.4; not in the UAT checklist)
D12	student	each gap's subjects shown in the order my degree intends, with the ones I have not sat yet marked, and the page saying which rule ordered them	I know where this weakness is assessed next and can prepare	R5	IOLG-106	Built, IOLG-106 branch (added in 0.5; not in the UAT checklist)

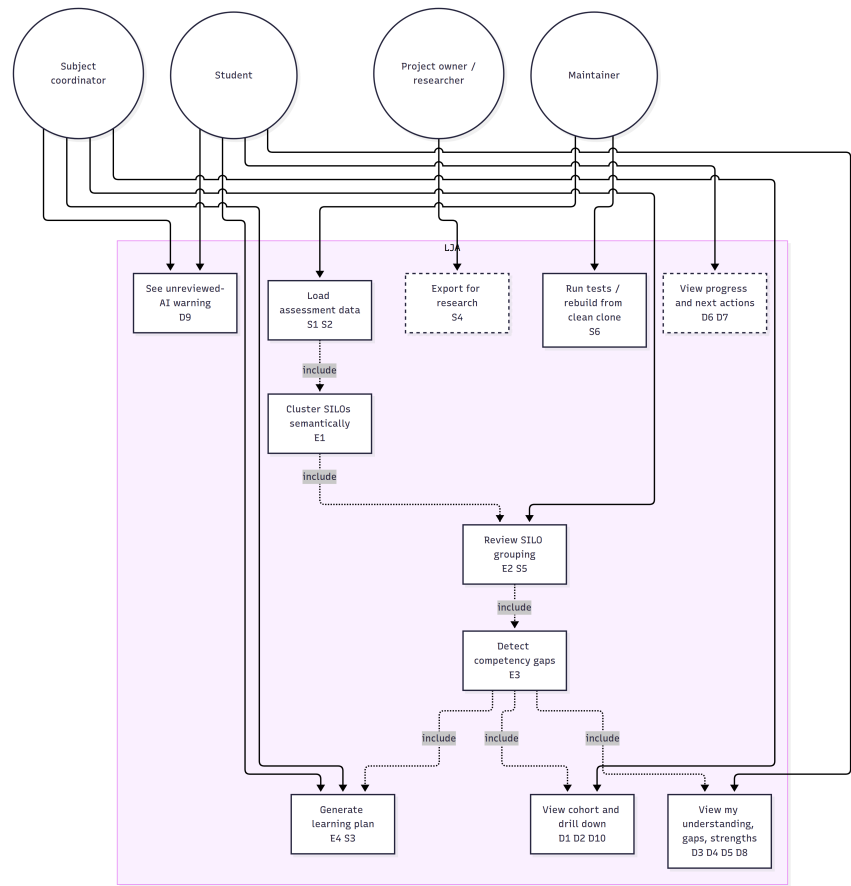
Adaptive quiz generation (R8) was descoped by plan and then built as a thin slice late in Sprint 5 (S8, PR #52). It is deliberately not a complete R8: see §3.10 and §8.1.

Layer	Choice	Version (pinned in requirements.txt)	Why
Language	Python	3.12	Team skills; the owner's brief asked for Python LLM integration; <code>psycopg2</code> , <code>pandas</code> , <code>openpyxl</code> cover both data paths
Data models	Pydantic v2 + frozen dataclasses	pydantic 2.13.4	Pydantic gives schema-validated LLM output (<code>extra="forbid"</code>); dataclasses keep the internal dataset immutable
Excel loading	<code>openpyxl</code> / <code>pandas</code>	3.1.5 / 3.0.5	Owner supplied <code>.xlsx</code> ; no need for anything heavier
Moodle access	Direct SQL over <code>psycopg2</code> , plus Moodle Web Services REST for discovery	2.9.12	Rubric fills are not exposed over Web Services (see §3.7 D3)
LLM, cloud	Anthropic SDK, model <code>claude-opus-4-8</code>	anthropic 0.121.0	Native structured output enforced server-side
LLM, local	Any OpenAI-compatible server (Ollama, LM Studio, llama.cpp), default model <code>qwen3-vl:30b</code>	openai 2.53.0	Zero-cost default; a fresh clone works without an API key
Web dashboard	FastAPI + Jinja2 + uvicorn	0.141.1 / 3.1.6 / 0.52.1	Small, read-only, server-rendered; no build step, no JS framework to maintain
Charts	Chart.js 4, plus d3 7 and d3-cloud 1 on the outcome-quality page, all from the jsDelivr CDN	4.x / 7.x / 1.x	Chart.js for every bar, line, scatter and bubble chart (with the Enlarge modal in <code>chartzoom.js</code>); d3 only for the word cloud and chord figure on /silos. Not vendored, so charts need internet (§5.7)
Catalogue and export extras	PyYAML; matplotlib and Jupyter (developer environment only)	6.0.3	The subject catalogue is <code>YAML docs/export-sample.ipynb</code> charts an export. Neither matplotlib nor Jupyter is in <code>requirements.txt</code> , so CI

Layer	Choice	Version (pinned in requirements.txt)	Why
			and a pip install do not need them
Dev Moodle	moodlehq/moodle-docker, Moodle 5.2, PostgreSQL	5.2.2 verified	Official tooling; local-first per the owner
Environment	conda (python/environment.yml) or pip (requirements.txt)	—	conda for developers; pinned pip for CI reproducibility
Lint / test	ruff 0.16.4, pytest 9.1.1, pytest-cov	pinned	ruff pinned so a new rule cannot redder an unrelated PR
Security scanning	gitleaks 8.18.4 (full history), pip-audit	pinned	Tender R9 names secret scanning as Mandatory
CI	GitHub Actions	—	Three independent jobs; branch protection enforced for admins

3.3 Use case diagram

Actors and use cases as drawn at 0.1. Dashed use cases were planned or in an open PR at the time; of those, strengths, progress, next actions, study strategies and the export are now on main, the clusters page is still PR #25, and the quiz (PR #52) and the priority-group, subject, assessment, /run and /glossary pages are not drawn at all. The source has not been re-rendered for 0.4.



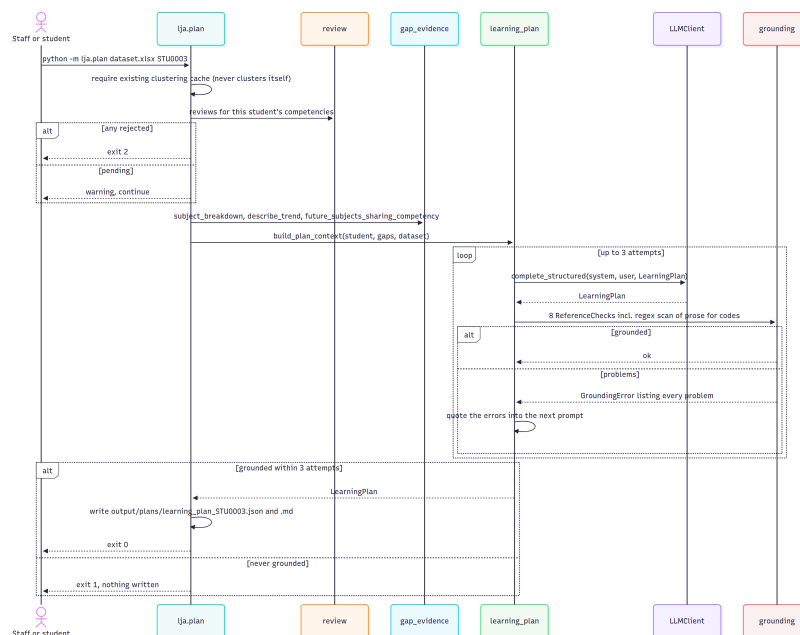
Use case diagram: actors and use cases, dashed = planned

Figure 2: Use case diagram: actors and use cases, dashed = planned. Source: docs/handover/diagrams/02-use-cases.mmd.

Note on actors: there is no authentication. The dashboard is bound to 127.0.0.1 and any viewer can open any student. “Student” and “subject coordinator” are roles the data is designed around, not roles the software enforces. See the threat model (§3.8).

3.4 Class diagram

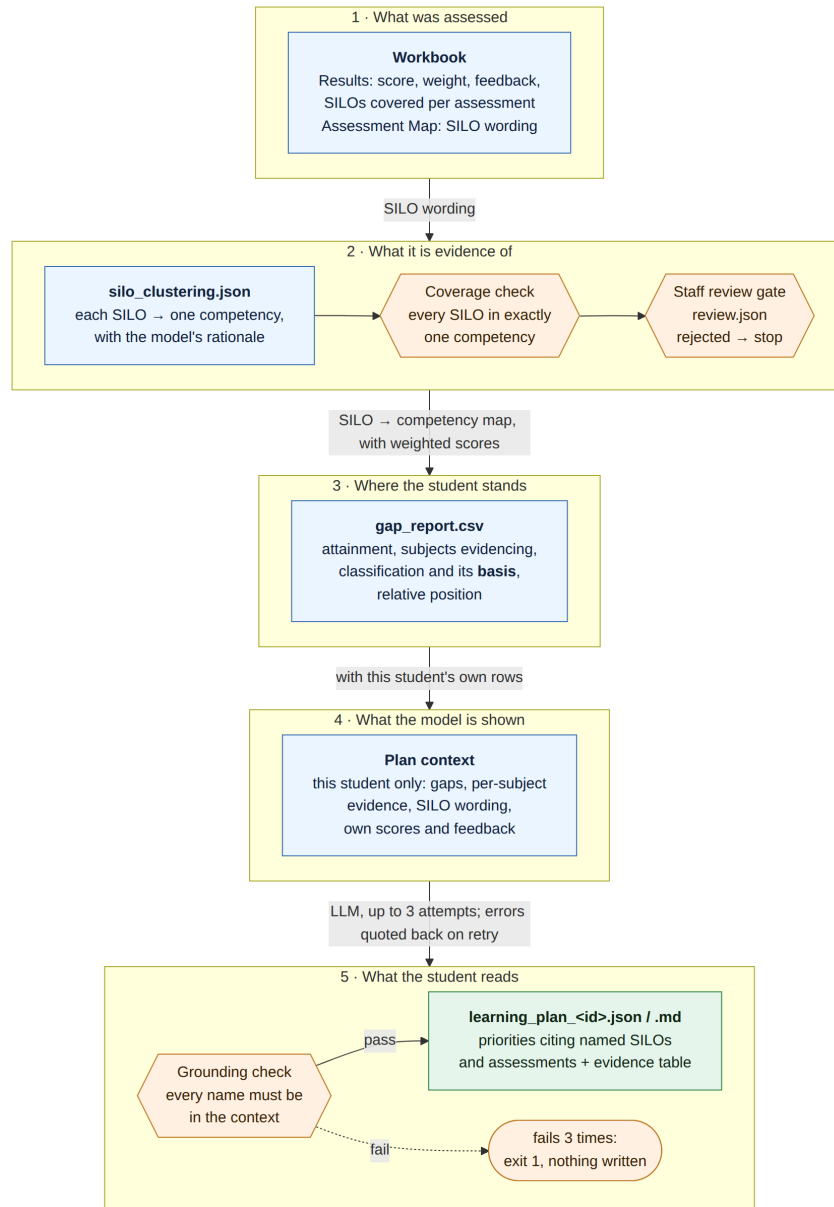
The classes below are the ones a maintainer will touch. Field lists are abbreviated; the source line for each is in the architecture inventory in §5.1.



Sequence diagram: generating a grounded learning plan

Figure 5: Sequence diagram: generating a grounded learning plan. Source: `docs/handover/diagrams/05-seq-Learning-plan.mmd`.

Tracing a plan's evidence (E4, requirements 5 and 6). Figure 5 shows the process. Figure 5b shows the evidence lineage instead: the file each stage leaves behind and the check between stages, so any statement in a plan can be followed back to specific marks, assessments and SILOs. The companion document, `docs/handover/learning-plan-traceability.md`, walks one real claim from the reference run back to the workbook. It also explains why this traceability matters educationally and what the trace exposed.



Evidence lineage of a learning plan

Figure 5b: Evidence lineage of a learning plan. Source: `docs/handover/diagrams/07-evidence-lineage.mmd`.

3.6 Data model and file formats

Input workbook (`data-fixtures/CSE_results_150_students_3_Subjects.xlsx`, supplied by the owner on 11 Aug 2026): three sheets.

Sheet	Columns (essential)	Becomes
Assessment Map	Subject, Assessment, Weight, Contribution, Early, Hurdle, SILOs (semicolon list, so SILO text must not contain ;)	Assessment, Silo
Results	Student ID, Subject, Assessment, Score, Feedback, Weight, Weighted score, SILOs	ResultRow
Student Summary	Student ID, per-subject totals, Average total, Performance band	StudentSummary

Moodle path: SQL Query 2 in `sql/moodle_attainment_extraction.sql` (templated on `{prefix}`) returns one row per (student, assessment, rubric criterion) with the level awarded, the criterion's maximum and the marker's remark. The loader joins those rows to a staff-editable CSV `criterion_silo_map_<SUBJECT>.csv` (subject, assessment, criterion_text, silo_key, weight). An unmapped criterion is a hard error. This mapping CSV **is the product**, not a workaround: Moodle's own Outcomes (Query 3) and Competency framework (Query 4) tables were both empty for the seeded subject.

Generated files (all under `python/output/`, gitignored):

File	Producer	Format	Notes
<code>silo_clustering.json</code> (<code>silo_clustering_moodle.json</code> for <code>--source moodle</code>)	<code>lja.cli</code>	<code>SiloClusteringResult</code> JSON	The LLM cache. Regenerable. Delete or <code>--refresh-clustering</code> to re-cluster.
<code>silo_clustering.review.json</code>	<code>lja.review</code> / <code>lja.cli</code>	<code>ReviewStore</code> JSON	Staff decisions. Not regenerable. Cluster ids are the first 12 hex chars of SHA-256 over the sorted member keys, so a re-clustering that produces the same membership keeps its decision.
<code>clusters.csv</code>	<code>lja.cli</code>	CSV	Cluster table plus SILO definitions
<code>gap_report.csv</code>	<code>lja.cli</code>	CSV	<code>student_id</code> , <code>competency_label</code> , <code>attainment_pct</code> , <code>subjects_evidencing</code> , <code>n_observations</code> , <code>classification</code> , <code>classification_basis</code> , <code>relative_position</code>
<code>plans/learning_plan_<id>.json</code> , <code>.md</code>	<code>lja.plan</code>	<code>LearningPlan</code> JSON, Markdown	Only written after grounding passes. The dashboard reads the JSON from <code>LJA_DASHBOARD_PLANS_DIR</code> for the next-actions section (D7)
<code>strategies/study_strategy_<id>.json</code> , <code>.md</code>	<code>lja.strategy</code>	<code>StudyStrategy</code> JSON, Markdown	One entry per gap; techniques from a closed list (§3.10). Only written after grounding passes
<code>quizzes/quiz_<id>.json</code> , <code>.md</code>	<code>lja.quiz</code> (PR #52)	<code>QuizDocument</code> JSON, Markdown	Questions plus the blind educator review (§3.10). The dashboard reads the JSON from <code>LJA_DASHBOARD_QUIZZES_DIR</code>
<code>export/students.csv</code> , <code>competencies.csv</code> , <code>cohort.csv</code> , <code>manifest.json</code>	<code>lja.export</code>	CSV, JSON; schema 1.0 in <code>docs/export-schema.md</code>	A projection of a run, never a re-decision. <code>--anonymise</code> pseudonymises ids with an HMAC keyed on <code>LJA_EXPORT_SALT</code>
<code><workbook>.truth.json</code> , <code>.clustering.json</code> , <code>.clustering.review.json</code>	<code>lja.data.catalogue_generator</code>	JSON	Beside a generated workbook: the answer key, the catalogue's competency tags in cache shape, and an all-confirmed review file. <code>/run</code> reads the seed and parameters from the truth file and deliberately never the answer key

Classification vocabulary (`gap_detection.py`): `classification` ∈ {persistent gap, isolated gap, developing, proficient}; `classification_basis` ∈ {relative position, absolute floor, absolute ceiling, insufficient data}. Persistent means evidenced in two or more subjects. The basis column exists for traceability (R5): every label says which rule produced it.

3.7 Design decisions and their justification

Each decision names the quality it serves. Scalability, flexibility, usability and limitations are addressed in each, as the Week 9 lecture asked.

D1. Standalone system beside Moodle, not a Moodle plugin (tender Option 2). The only option that delivers every must-have inside a semester while keeping identifiable data out of scope. The extraction layer is behind an interface, so Option 3 (a plugin that exposes rubric fills as a web service) stays open. *Flexibility.* Limitation: an institution has to run a second service.

D2. Two data paths into one LjaDataset. The Excel path exists because the owner supplied a ready-extracted workbook and the team could build the algorithmic core without waiting on Moodle plumbing. The Moodle path is the production path. Proof that the abstraction holds: `--source moodle` was added with no change to clustering or gap detection. *Flexibility, scalability of the team.* Limitation: the Moodle loader is proven on one subject with five students; multi-subject extraction is future work.

D3. Rubric fills are read from the database, not Web Services. Stated plainly, as the owner asked. Moodle's Web Services expose a rubric's *definition* (criteria and levels) but no function returns which level a marker *awarded* or their per-criterion remark. Those live in `gradingform_rubric_fillings`, so the production path connects to PostgreSQL as a read-only role and runs Query 2. Consequences for an operator: LJA needs a database credential, not just a token; the connection must be to a replica or with a role that can only `SELECT`; and any Moodle

upgrade that changes the `grading_instances/assign_grades` join must be re-verified (§5.8). *Limitation accepted deliberately.*

D4. Provider-agnostic LLM layer, defaulting to a free local model. One `LLMClient` protocol, two native implementations. Feature code never imports `anthropic` or `openai`. The default is `openai_compatible` so a fresh clone runs with no API key. Justification from the tender: a governance decision about which provider the university permits “changes configuration rather than architecture”. *Flexibility.* Limitation: local model quality varies enormously (§3.9).

D5. Direct LLM clustering rather than an embedding pre-pass (ADR 0002, accepted 20 Sep 2026). With 13 SILOs an embedding model, vector store, similarity threshold and retrieval stage add moving parts without adding accuracy. This is a recorded deviation from the tender, which promised “embeddings with LLM adjudication”. The ADR names the reconsideration triggers: SILO count grows substantially, prompt size or latency become a problem, or consistency stays inadequate. The 52-SILO run on 20 Sep hit the first trigger (§3.9), so this decision is due for review. *Simplicity now, scalability later.*

D6. Grounding is code, not prompt. The prompt forbids inventing names; the grounding module (`lja/llm/grounding.py`) checks the response against the input vocabulary and raises. Three categories: `unknown` (always), `missing` (when completeness is required, as for clustering), `duplicate` (when uniqueness is required). Names are compared exactly after `strip()`, no case folding, no aliasing, because a fuzzy match is how a hallucination gets through. Plans fail closed: three attempts, errors quoted back, then exit 1 with nothing written. *Usability for the student, who can trust every name; limitation is that bare `SIL01` ids in prose pass as ambiguous.*

D7. Staff gate before any gap report. Clustering output is treated as `mapped_by='llm'`, `confirmed_by_staff=False` until a person says otherwise. Decisions live in a separate `.review.json` so regenerating the cache never discards them. The CLI blocks (exit 2) on any rejected cluster, and on pending ones unless `--allow-unconfirmed` is passed; the dashboard shows a banner. The team settled “block, don’t warn” by merging PR #8. *Usability for staff; a limitation is that the review is CLI-only, there is no confirmation UI.*

D8. Relative gap detection with median and MAD (ADR 0001). The owner’s primary signal is “variability within a discipline, not raw failure”. Position = (attainment – student’s median) / MAD. Median and MAD rather than mean and SD because a student carries roughly 4–8 competencies and at that n one catastrophic result drags the mean far enough to hide everything else. Absolute floor (50) and ceiling (75) are checked first so a uniformly weak student still gets gaps and a uniformly strong one gets none. Fallbacks (too few competencies, flat profile) are recorded in `classification_basis`. **Every threshold is an environment variable and every default is a documented proposal, not a ratified value:** the owner confirmed there is no institutional at-risk number. *Usability for the student; limitation is that the supplied dataset is nearly flat (§3.9).*

D9. Dashboard is read-only and never calls the LLM on a page load. A page load must never trigger a billed API call. The app is a `create_app(dataset, gaps, clustering, review_warning)` factory; everything is computed at start-up from the cache. Statistics are population statistics computed in `stats.py`; sorting is progressive enhancement in 83 lines of vanilla JS. *Usability, cost control.* Limitation: no auto-reload, no auth, Excel source only. D20 adds the one deliberate exception.

D20. Generation from the student page is a button, opt-in, and a subprocess of the same command (IOLG-137). Where a plan or quiz is missing the page used to print the command to type; the project owner’s reviewer asked for a button. The button exists only when the dashboard is started with `--allow-generate` (or `LJA_DASHBOARD_GENERATE=1`), because the dashboard has no login and a control that spends LLM calls must be an operator’s choice; the Provenance page says whether it is on. A press is an explicit `POST`, never a page load, and it runs `python -m lja.plan` or `lja.quiz` for that one student with the workbook, cache and review file the dashboard was started from, into the directories it reads. There is no second code path to a model, so the gate, the grounding and the fail-closed rule are the CLI’s own. The page polls the job and shows the command’s output lines (each grounding attempt), then re-renders from the file; the CLI writes only after grounding passes, so a failed run leaves the previous file untouched. One job per student and artefact at a time. *Usability for staff; D9’s cost rule kept.* Limitation: no queue, no cancel, and anyone who can reach the port can press the button, which is why it is off by default and the dashboard binds to localhost.

D10. Generated artefacts are separate commands. `lja.plan` is not part of `lja.cli` so that re-running the pipeline never silently re-spends plan calls. *Cost control.*

D11. The “At Risk” cohort is deliberately absent. `/cohort/at-risk` returns 404 and a test asserts that. An at-risk rule would be a policy decision the owner has not made; shipping one would look like one. *Limitation by design.*

D12. CI gates are honest about what they enforce. Coverage is reported, not gated (the 80% tender target is met on the core modules but not on two CLI entry points); `pip-audit` reports but does not block (its result depends on the CVE feed, not the diff); line length is measured but not enforced. Each is written down with the condition for tightening it. *Maintainability: “a threshold that fails on the day it lands is a threshold somebody deletes by the end of the week.”*

D13. Assessment score counts as full evidence for every SILO it addresses. Weighted only by the assessment’s weight. This is an approximation, flagged for the owner, because the workbook does not carry per-SILO marks within an assessment. The Moodle path’s per-criterion rubric fills are the way past it. *Limitation.*

D14. Priority groups combine classifications the pipeline already made; they add no threshold (IOLG-134). On a cohort with real per-competency spread the relative rule flags most students (4,195 of 5,000 on the 100-subject cohort, but only 1,145 with anything under the 50% floor), so a single “persistent gap” tile misleads. The dashboard splits flagged students into 1st (at least one competency below the absolute floor), 2nd (no floor breach but a persistent gap) and 3rd (isolated gaps only), ordered within each group by the student’s lowest flagged mark. Splitting further would need a cut-off nobody has ratified, which is action A-01, not a dashboard default. The ordering is computed from the classifier’s outputs alone; the dashboard never reads a generator’s answer key, and

on the synthetic cohort no such ordering surfaces the planted gaps (62 of 371 in the top 371), which is a finding about the method rather than something the page can fix. *Usability for staff; honesty about what a count means.*

D15. Every count on the dashboard is a link to the list it counts, and every page states its own provenance.

The `tile` macro is the only way a count is rendered, each list page decides membership in `app.py` and a test asserts the list holds exactly what the tile counted. `/run` shows the command, git hash, inputs and every threshold with its value, default, variable and flag; `/glossary` defines each term once with this run's values. *Traceability (R5).* Limitation: `/run` can only describe the run it is serving, because `create_app()` takes the same `GapThresholds` that `compute_gaps()` was given.

D16. Generated artefacts are a family with one pattern, and the dashboard only ever renders them.

`lja.plan`, `lja.strategy` and `lja.quiz` share `build_plan_context()`, the same `--source`, cache and review-file options, the same staff gate (rejected cluster exits 2, pending warns) and the same fail-closed loop (never grounds, exit 1, nothing written). Strategies and quizzes ground *per competency*, stricter than plans: an entry's subjects, assessments and SILOs must belong to the competency it is filed under. The dashboard reads their JSON and never generates. *Cost control, and one discipline to maintain instead of three.* §3.10 describes each.

D17. The export pseudonymises with a keyed HMAC and refuses to run without a key. `--anonymise` maps each student id to an HMAC-SHA256 pseudonym keyed on `LJA_EXPORT_SALT`, so the same student lines up across exports taken with the same salt while the mapping cannot be reversed without it. An empty salt exits 2 rather than keying on "", which would be stable but guessable. `manifest.json` records source, git commit, cache and the seven thresholds so two exports can be diffed knowing they came from the same pipeline. *Privacy (owner NFR-2); research use (R7).* Limitation: the export has no staff-gate check of its own (compliance item SE-5).

D19. Progress is order, not time, and the order is declared (IOLG-106). The workbook has no dates, so the only honest meaning of progress is position in the intended subject sequence. That sequence is configuration (`LJA_SUBJECT_SEQUENCE`), where the owner's course map (FR-1.7) goes when supplied, with the year digit in the subject code as the documented fallback and "unknown" when neither applies. Every trajectory carries a `basis` saying which rule ordered it, as every gap carries a `classification_basis`, and the stable band that separates improving from stable from declining is `LJA_TREND_STABLE_BAND`, shown on `/run` with the gap thresholds. The subjects ahead of a student are listed as where the competency is assessed next and labelled "prepare", never "avoid": subject choice is an academic decision whose rules the system does not hold. *Traceability (R5); the tender guardrail that mastery estimates are formative.* Limitation: two or three points per competency is an ordering, not a slope, and nothing here extrapolates.

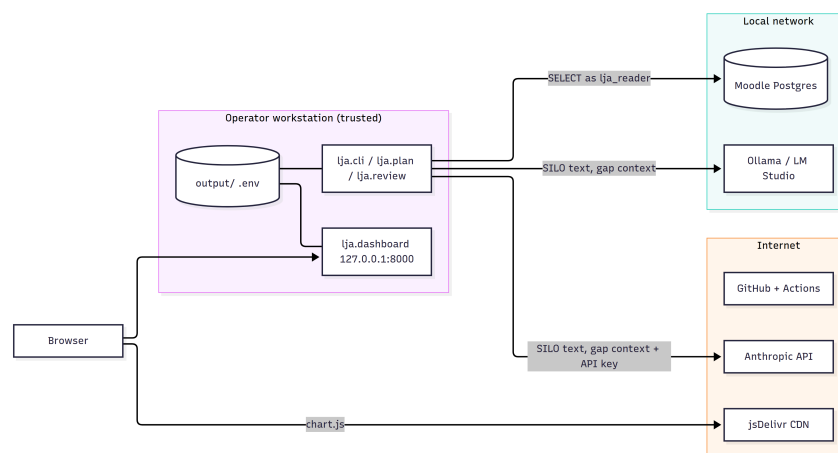
D18. Embeddings are used for the catalogue's competency tags, not yet for the pipeline's clustering. The 52-SILO failure (§3.9) hit ADR 0002's review trigger. The answer built in IOLG-113 is `competency_tagger.py`: embed every SILO (`nomic-embed-text` through the OpenAI-compatible endpoint), spherical k-means, and ask the chat model only to label each cluster in batches of 20. It tagged 1,436 real SILOs in 64 seconds. It produces the generator's ground truth; `cluster_silos()` in the pipeline is still a single LLM call. Swapping the pipeline over is the deferred decision in §8.2. *Scalability, deliberately staged.*

3.8 Threat model

△ *DRAFT.* This section is the team's first STRIDE pass over the system's trust boundaries. Since 0.3 the six security checks of IOLG-111 have been run and recorded in `docs/security-evidence.md` (28 Sep 2026, dev instance), and the risk register has been re-scored (`docs/risk-register.md`); the table below reflects both.

Assets. (1) Student assessment data, synthetic today but real if the system is deployed. (2) The Moodle database credential and web-service token. (3) The LLM API key. (4) The staff review file, because a tampered review file silently opens the gate. (5) Generated plans, which students act on. (6) The repository and CI, because a compromised dependency runs in every developer's environment.

Trust boundaries.



Trust boundaries: workstation, local network, internet

Figure 6: Trust boundaries: workstation, local network, internet. Source: `docs/handover/diagrams/06-trust-boundaries.mmd`.

Threat (STRIDE)	Where	Control in place	Gap / recommendation
Spoofing a viewer	Dashboard	Bound to 127.0.0.1 by default; no accounts	No authentication. Never bind --host 0.0.0.0 on a shared network with real data. Auth is future work.
Denial of service or cost by pressing Generate	Student page (IOLG-137)	Off unless --allow-generate; one job per student and artefact; Provenance page states it is on; the CLI's own gate and attempt limit	With it on, anyone reaching the port can start LLM calls. Keep it off on anything but localhost; add auth before exposing it.
Tampering with the review file	output/*.review.json	Separate from the cache; state machine forbids flipping confirmed → rejected directly	File is plain JSON with no signature. Keep output/ on an operator-only path; consider a hash in the file.
Tampering with Moodle	Postgres	Role lja_reader is SELECT-only; nothing in the code issues a write; fixtures mark via assign::save_grade()	Observed on the dev instance, 28 Sep (IOLG-111): role created from the sql/README.md DDL; UPDATE m_user as lja_reader returned permission denied; a pg_dump --data-only checksum was identical before and after lja.cli --source moodle (21,554 data lines byte-identical). Not yet applied on a shared or production instance.
Repudiation of a staff decision	Review file	note field; git history if committed	No reviewer identity or timestamp is stored. Add reviewed_by and reviewed_at.
Information disclosure to the LLM provider	Anthropic path	Prompts carry SILO text, gap classifications, assessment names and marker feedback, never names beyond STU0001-style ids	With real data, ids plus per-subject scores are re-identifiable. Pseudonymise at ingest before any cloud provider is used (owner NFR-2, FR-1.5). Prefer the local provider for real data. Note the plans, strategies and quizzes send the student's own marker feedback to the model; with real data that is bespoke text.
Information disclosure in the research export	lja.export	--anonymise HMAC pseudonyms keyed on LJA_EXPORT_SALT; empty salt refused; cohort.csv carries no ids	The salt must stay out of git and out of tickets: whoever holds it can re-identify every export. students.csv still carries per-subject totals, which are quasi-identifiers on a small cohort.
Information disclosure via secrets	.env, CI	.env gitignored; gitleaks over full history, blocking; workflow permissions contents: read	Rotate any token that was ever pasted into chat or a ticket.
Information disclosure via logs	CLI output	Usage summaries print token counts, not content	Plans print to stdout in .md; do not pipe to shared logs with real data.
Denial of service by a hung LLM server	OpenAI-compatible client	Three response_format strategies each with a 600 s timeout	A hung server can burn 30 minutes and misreport as a JSON error (action A-30). Add LJA_OPENAI_TIMEOUT.
Elevation of privilege via dependencies	requirements.txt	Pinned versions; pip-audit in CI	pip-audit is non-blocking. Triage and flip continue-on-error to false (§5.4).
Supply chain via the CDN	base.html	None	Chart.js is fetched unpinned from jsDelivr at page load. Vendor it under static/ and pin the version (A-20).
Misuse of threshold flags as a grade-changing tool	CLI --absolute-floor etc.	Output says which basis produced each label; /run shows every threshold used, with changed values marked	If thresholds become viewer-adjustable (A-27), label it as sensitivity exploration, not policy.
Unverified content reaching a student	lja.quiz (PR #52)	Grounding checks every reference; a blind second pass answers each question without the key and the code compares; the page says the key is unchecked and asks for staff review	Nothing in the pipeline can check that a marked answer is correct (§3.10). "For educator view only" is a label, not an access control: the dashboard has no logins. Do not show quizzes to real students before a person has reviewed them.
Web Services token scope	moodle_probe.py	Four functions needed	IOLG-111 check 5: the dev token (moodle_mobile_app

Threat (STRIDE)	Where	Control in place	Gap / recommendation
			service) authorises 429 functions. The production path does not use Web Services, but if one is ever adopted, define a purpose-built service limited to those four.

3.9 Known limitations

1. **The supplied dataset is nearly flat.** Measured MAD across the 150 profiles: min 0.00, median 0.90, max 3.80. Each synthetic student is one baseline plus independent noise with no per-competency ability term, so relative detection has little to find. This is probably an artefact of how the workbook was generated, not a fact about students. **Do not tune LJA_GAP_MIN_SPREAD down to make more gaps appear.** The catalogue generator (IOLG-113) adds per-competency ability precisely so the detector can be evaluated.
2. **The relative detector flags almost everyone once profiles have real variance, and the cutoff now has a measured sensitivity curve.** On 500 generated students at the default -1.0 MAD cutoff, 448 had a persistent gap and 32 of 33 planted gaps were recovered. On the 3,000-student handbook cohort (314 subjects, 1,436 real SILOs) the same run gave 227 of 245 planted gaps found (93%) with 2,240 students flagged at -1.0; 220 (90%) and 1,833 at -1.5; 207 (84%) and 1,420 at -2.0, and in each case over 90% of the unplanted flags were in the student's genuinely weakest third. The detector is accurate about *which* competency is weak; the cutoff trades a little recall for far fewer flags. That table is the input action A-01 was missing. The priority groups (D14) are the dashboard's answer in the meantime.
3. **Local LLM quality is the biggest single risk.** An 8B model grouped SILOs by subject (semantically useless). A 30B model did genuine cross-subject grouping but dropped SILOs on some runs, which is why coverage validation and retry exist. Run-to-run variance means clustering is not reproducible; the cache and the review file are what make a run stable.
4. **Single-call clustering does not scale.** At 52 SILOs (a realistic multi-subject catalogue) the 30B model failed coverage on all three attempts and was aborted after 7m43s. The embedding tagger (D18) shows the alternative works at 1,436 SILOs, but it is wired to the catalogue, not to `cluster_silos()`. Options for the pipeline remain: a stronger model, chunking by year level or subject pair with a merge pass, a repair pass, or the embedding pre-pass (ADR 0002's review trigger).
5. **Rubric fills need database access** (D3). Only the Moodle path is affected.
6. **Moodle loading is proven for one subject** (CSE1IOI, 5 students × 3 criteria), and the loader hardcodes assessment weight 1.0 and no early/hurdle flags.
7. **No timestamps in the workbook**, so "progress" means position in the declared subject sequence (D19), not time. On the three-subject fixture every student has sat all three subjects, so no gap card shows a subject ahead; the "ahead" case appears on the generated cohorts.
8. **Charts need internet** (CDN). **No auth. No confirmation UI** (review is CLI-only). **Dashboard reads Excel only** (no `--source moodle` there yet).
9. **Feedback text is templated:** 45 unique strings across 1,650 rows. Feedback analysis features were not built for that reason; the owner flagged that bespoke feedback carries re-identification risk.
10. **Docs drift.** Parts of `python/README.md` predate the review gate and the plan command's review awareness (§5.9 lists what to fix).
11. **The quiz's answer key cannot be checked** (PR #52). The only subject matter in the input is the SILO wording, assessment names, marker feedback and handbook synopsis; the question, key and distractors come from the model's general knowledge. The blind second pass catches disagreement, not shared blind spots: on the first live run the same model agreed at high confidence with a composition-versus-aggregation question a tutor could reasonably mark the other way. See §3.10.
12. **The review banner names a count, not the cluster** (UAT-02). With a rejected cluster the dashboard says "1 AI-generated SILO cluster(s) have been rejected by staff" while the CLI names it. The S5 acceptance criterion fails on the dashboard path until the banner is amended or the criterion revised.
13. **The quick start leaves the default cache path empty** (UAT-01). Step 3 reads the committed reference cache with `--clustering-cache` but writes nothing to `output/silo_clustering.json`, so the bare `lja.plan` and `lja.export` examples exit 2 until the same flag is carried through. A documentation fix, routed to IOLG-114.
14. **Prose in generated plans can misattribute.** The ten-plan audit passed every structured check, but the assisted pre-review found prose concerns in eight plans: feedback attributed to the wrong assessment, a summary counting two persistent gaps where there is one, "steady progress" with no measured improvement. Grounding checks names, not claims. Human sign-off is pending (IOLG-121).

3.10 Generated artefacts: plans, strategies and the practice quiz

Three commands generate something for one student. They are one family (D16): same inputs, same gate, same fail-closed loop, and the dashboard only renders what they wrote. What differs is what each one is for and what its grounding rules add.

	<code>lja.plan</code> (IOLG-108)	<code>lja.strategy</code> (IOLG-123)	<code>lja.quiz</code> (IOLG-136, PR #52)
Answers	<i>What</i> to work on	<i>How</i> to study it	<i>Whether</i> the gap is closing
Output	<code>LearningPlan</code> : priorities with evidence and actions	<code>StudyStrategy</code> : one entry per gap	<code>QuizDocument</code> : questions per gap, plus the educator review
Extra input	—	—	The subject catalogue, for titles and handbook synopses (<code>--catalogue</code> ; optional)
Grounding beyond the plan's	—	Per competency: subjects, assessments and SILOs must	Per competency as for strategies; every gap gets

	lja.plan (IOLG-108)	lja.strategy (IOLG-123)	lja.quiz (IOLG-136, PR #52)
		belong to the competency the entry is filed under; exactly one entry per gap, none for a strength; gap_kind matches the engine	exactly --items-per-gap questions; options 3 or 4, distinct, valid key; written tasks carry 2 to 5 marking points; no invented code in any prose field
Structural rule checked in code	—	Persistent gap: name two or more evidencing subjects and use interleaving or spaced practice. Isolated gap: include feedback review of the named assessment	A SILO whose wording contains a doing verb (implement, design, analyse, ...) must get a written task under the default mixed policy
No gap	A plan is still written	One line, exit 0, no LLM call	One line, exit 0, no LLM call
Shown on the dashboard	Next actions on the student page (D7)	Not yet	Student page, with synopses and collapsed educator notes (PR #52)
Tests	test_learning_plan.py (17), test_plan_cli.py	test_study_strategy.py (19), test_strategy_cli.py (6)	test_quiz.py, test_quiz_cli.py (74 tests on the branch)

Study strategies. Techniques are a closed list (STUDY_TECHNIQUES): the six strategies in Weinstein, Madan and Sumeracki (2018), *Teaching the science of learning* (spaced practice, retrieval practice, interleaving, elaboration, concrete examples, dual coding), plus worked examples and feedback review. The schema rejects any other name, so the model cannot recommend something plausible with no evidence behind it. First live run on 27 Sep (qwen3-vl:30b): three students, every strategy grounded on the first attempt, 20 to 97 seconds each.

The practice quiz, and why it is a thin slice. The tender listed the adaptive quiz last and named it the first descope candidate, because a quiz is the artefact “most likely to look plausible while quietly not being grounded”. The binding plan cut it, and the slice in PR #52 was built late in Sprint 5 so the next team starts from a grounded, tested baseline rather than nothing. It does four things:

1. **Two kinds of question, and the educator decides the policy.** A `multiple_choice` item has 3 or 4 options and a key; a `written` task has a stem asking the student to produce something, a `model_answer` a tutor would accept, and 2 to 5 `marking_points`. Retrieval practice works best when the student generates the answer, so written tasks are the better instrument where the SILO asks for doing; multiple choice is the only kind that self-marks. --format is `multiple_choice`, `written` or `mixed` (default). Under `mixed` the model chooses per question inside one rule the code also checks (`requires_written()`): a doing-verb SILO must get a written task. The prompt tags those SILOs “(written task required)” so the model and the validator read the same rule.
2. **A blind educator review as a second pass.** After the quiz grounds, one more call (skip with `--skip-educator-review`) gives the model every question with its options, SILO wording, assessment and synopsis but **not** the key, and asks it to choose, state a confidence, write the explanation a tutor would give and flag concerns. The code, never the model, compares the choice with the key. For a written task the pass acts as a second marker against the model answer and marking points (`meets`, `partly`, `fails`). The result is stored as `educator_review` and rendered under each question in a collapsed block labelled “For educator view only”, with a summary line naming any disagreement. A disagreement does not prove either side wrong; it says which question to check first. A review that never grounds is a warning, not a failure: the quiz is still written, without notes.
3. **Handbook synopses.** The crawl always parsed each subject's handbook description; it now lands in the catalogue as `Subject.description`. The three supplied subjects have theirs from the cached 2026 pages; synthetic subjects have none and the page says so rather than inventing one.
4. **It says what it cannot check.** The page and the Markdown both state that the answer key is unverified and ask for staff review before a student sees the quiz.

First live runs, 2 Oct 2026, qwen/qwen3-vl-30b via LM Studio on the reference run, STU0003 (one persistent gap, one isolated): multiple choice only grounded on the first attempt in 17 s, four questions; the mixed policy produced four written tasks (every gap SILO for this student is a doing SILO) with five marking points each, grounded on the second attempt because the first omitted the required `kind` field; the second-marker pass found every model answer meets its marking points. Both review calls were under 10 s. The limit showed up immediately: a question marking “composition” for a car and its wheels, with “aggregation” as the tempting distractor, which a tutor could reasonably mark the other way, and the blind review by the same model agreed with it at high confidence. A model reviewing its own questions shares its own blind spots, so point the reviewer at a different model family where one is available.

Closing the gap means one of two things, neither built: feed real course material (lecture notes, tutorial questions, rubric criteria text) so a question can be checked against a source, or run the blind pass on an independent model and reject on disagreement. Until then the quiz is a staff tool, not a student one.

3.11 Trajectory: a competency across the subject sequence

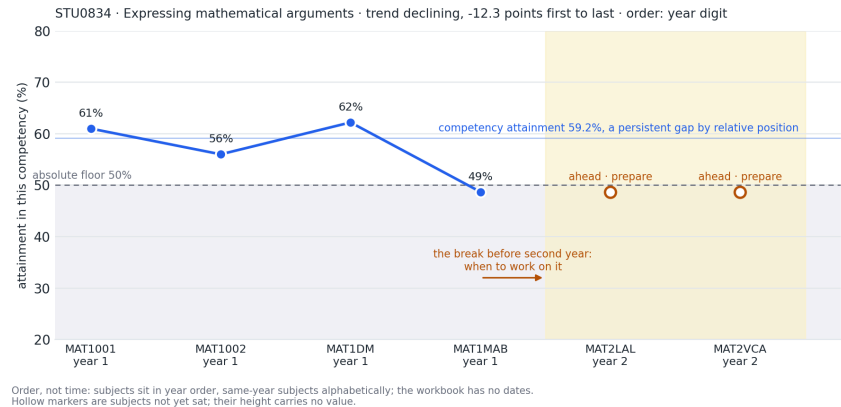
`lja/model/trajectory.py` (IOLG-106) places the per-subject evidence behind one competency verdict in the order the degree intends the subjects to be taken, with the subjects the student has not sat yet ahead of them. It answers the two questions a gap card raises: is this weakness recurring as the student moves through the sequence, and where is it assessed next. It is the thing the owner described on 11 August as “gap in first year, consequence in third year”.

Type	Holds
SubjectSequence	The declared order, parsed from LJA_SUBJECT_SEQUENCE. <code>position(code)</code> returns a <code>SequencePosition</code> with a <code>source</code> : <i>declared sequence</i> for a listed subject, <i>year digit</i> for one placed by the digit in its code, <i>unknown</i> otherwise. Declared subjects sort first, then year-digit ones by year, then unknown ones alphabetically.
TrajectoryPoint	One subject in the competency: its source, year level, whether the student has sat it, the per-subject attainment (the same figure as the gap card's evidence table) and observation count.
Trajectory	The points in order; <code>delta</code> (last ordered subject sat minus first, one decimal place) and <code>label</code> (improving, stable, declining, insufficient evidence); <code>basis</code> (which source ordered the subjects sat: declared sequence, year digit, mixed, none); the band in force. <code>taken</code> and <code>ahead</code> are the two halves.

`compute_trajectory(dataset, clustering, student_id, competency_label)` builds one;
`compute_trajectories()` builds them for a list of (student, competency) pairs, the gap report's own grain.
`describe_trend()` in `gap_evidence.py`, which the dashboard, the plan context, the export and the outcome-quality progressions all call, is now a thin alias over the same ordering and band, so every trend word on every page and in every file is computed one way.

Rules. A subject the student has sat but with no result touching this competency's SILOs is *taken*, not ahead. An unknown student has nothing ahead. Only ordered subjects take part in the trend; fewer than two is *insufficient evidence*. The band test is strict: a delta exactly equal to the band is stable.

On the dashboard. Each gap card carries the subject chain under the trend line: taken subjects as plain chips with their attainment, subjects ahead as amber dashed chips marked “ahead · prepare” (the marker appears only on a gap, because “prepare for” is advice about a weakness, not a remark about a strength), and an “order:” note giving the basis. The existing “flag for intervention” sentence now lists the subjects ahead in sequence order and says the first is where it comes up next. The progress table's columns follow the same order. `/run` has a “Subject order for progress and trends” table showing the declared sequence, the band, and how many of this run's subjects fell back to the year digit; `/glossary` defines *trajectory* and *subject sequence*.



A trajectory: STU0834 in Expressing mathematical arguments

Figure 5c: One trajectory from the 100-subject cohort, drawn for the User Document (its §4.5 explains it for readers of the dashboard). Four first-year subjects sat and declining, two second-year subjects ahead, ordered by year digit because the default declared sequence covers only the three supplied subjects.

Ahead is unfiltered without a course map. On the three-subject workbook “ahead” is exact. On the 100-subject cohort a competency can span fifteen subjects across seven programs, so a student's chain lists subjects from degrees they are not in, and a skipped earlier-year subject appears as ahead. The chain is correct about where the competency is assessed and wrong as a list of the student's next subjects; filtering it needs the program, which the catalogue's `programs`: `block` and the Student Summary's `Program` column both carry but the loader does not yet read. That is the first follow-up (§8.2).

Not a prediction. With two or three points per competency the trajectory is an ordering, not a slope, and nothing extrapolates. The subjects ahead are where the competency is assessed next, which is the moment to prepare; they are never advice about what to enrol in (D19). Enrolment data from Moodle would make “ahead” a fact rather than an inference from the program; that is a loader extension, not built.

Tests: `test_trajectory.py` (17): sequence parsing, declared order beating the digit, the fallback and the unknown case, band boundaries, the band as configuration, a student who skipped a subject, each basis value, taken-without-result, unknown student. `test_dashboard.py` adds five: chain order and the ahead marker, the year-digit basis when nothing is declared, no marker on a strength, `/run` and `/glossary`.

4. Deployment and environment

4.1 Hardware requirements

Configuration	Minimum	Notes
Excel pipeline + dashboard + tests, LLM elsewhere	2 cores, 4 GB RAM, 1 GB disk	Any laptop. The full test suite runs in under 2 s.
Local LLM (qwen3-vl:30b via Ollama)	19 GB disk for weights; 24 GB unified/GPU memory recommended, or 32 GB system RAM on CPU (slow)	Verified on the team's workstation; an alternative is LM Studio on another LAN machine, pointed at by <code>LJA_OPENAI_BASE_URL</code> . An 8B model fits in 8 GB but fails the clustering task.
Moodle development environment (Docker)	4 GB RAM for the containers, 5 GB disk	Only for the Moodle path. First bootstrap ~10 min (downloads Moodle source and images).
Free-tier cloud VM (Oracle Always Free, Azure for Students)	4 GB RAM	Sufficient for Docker Moodle plus the pipeline; documented in <code>sprint5-briefs/00-background.md</code> §6.

4.2 Software requirements

Component	Version	Required for
Python	3.12 (CI pins it; <code>environment.yml</code> pins it)	Everything
conda (Miniforge or Miniconda) or pip + <code>libpq-dev</code>	any recent	Environment. <code>psycopg2</code> compiles against <code>libpq</code> , so on pip installs you need <code>libpq-dev</code> (Debian/Ubuntu) or <code>postgresql</code> headers.
Git	any	Clone
Docker Engine + Compose v2, or Docker Desktop (macOS)	Docker 24+	Moodle path only
Ollama, LM Studio or another OpenAI-compatible server, or an Anthropic API key	Ollama 0.3+	Clustering and plans. Not needed to run the dashboard from an existing cache (PR #22 commits one).
OS	Ubuntu 22.04+ and macOS 14 (Apple Silicon) verified; Windows via WSL2 expected to work, unverified	
Browser with internet	any modern	Dashboard charts (CDN)

4.3 Installation from a clean clone (Linux)

△ The clean-machine rebuild by a non-author (IOLG-118) has still not been performed; `docs/rebuild-check.md` (28 Sep) records why and four preflight findings: the Ubuntu conda prerequisite is implicit, the quick start's cache path does not match the later bare commands (UAT-01, §3.9 item 13), the Python README's setup and dashboard examples silently require a model, and a pip install needs `libpq-dev`. Treat these steps as the author's account, and log every deviation on IOLG-114.

```

# 1. Clone
git clone https://github.com/yetanotherpassword/LearningJourneyAssistant.git
cd LearningJourneyAssistant

# 2. Python environment (conda route)
cd python
conda env create -f environment.yml      # creates env "lja", Python 3.12
conda activate lja

# (pip route instead of conda)
# sudo apt install -y python3.12-venv libpq-dev
# python3.12 -m venv .venv && source .venv/bin/activate
# pip install -r ../requirements.txt

# 3. Configuration
cp .env.example .env
git check-ignore .env                  # must print ".env"
# Edit .env: choose LJA_LLM_PROVIDER and fill in the matching block (see 4.4)

# 4. Prove the install
python -m pytest -q                    # expect: all passed, 1 skipped (live Moodle
test)
ruff check .                          # pip install ruff==0.16.4 if missing; expect no
findings

# 5. Local LLM (skip if using Anthropic, or if a clustering cache is already present)
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3-vl:30b

# 6. Run the pipeline on the supplied dataset (S1)
python -m lja.cli ../data-fixtures/CSE_results_150_students_3_Subjects.xlsx
# First run calls the LLM and writes output/silo_clustering.json (minutes on a local
model).
# It will then stop with exit 2: the clustering is pending staff review.

# 7. Review the clusters (E2)
python -m lja.review                  # interactive; or --cluster <id> --state
confirmed
python -m lja.cli ../data-fixtures/CSE_results_150_students_3_Subjects.xlsx
# Now writes output/clusters.csv and output/gap_report.csv

# 8. Dashboard (D1-D10)
python -m lja.dashboard               # http://127.0.0.1:8000

# 9. One learning plan (S3), then a study strategy (S7) and the research export (S4)
python -m lja.plan ../data-fixtures/CSE_results_150_students_3_Subjects.xlsx STU0003
# writes output/plans/learning_plan_STU0003.{json,md}
python -m lja.strategy ../data-fixtures/CSE_results_150_students_3_Subjects.xlsx STU0003
# writes output/strategies/study_strategy_STU0003.{json,md}
python -m lja.export ../data-fixtures/CSE_results_150_students_3_Subjects.xlsx --out
output/export
# writes students.csv, competencies.csv, cohort.csv, manifest.json; no LLM call
instead of
# calling the LLM, pass the same flag to plan, strategy and export, or they exit 2 (UAT-
01).

```

macOS (Apple Silicon). The root README.md has a verified walkthrough: Docker Desktop and Miniforge via Homebrew, conda init zsh, then the same steps. **Windows.** Use WSL2 with Ubuntu and follow the Linux steps; Docker Desktop with the WSL2 backend for Moodle.

Moodle development environment (S2), optional.

```

sudo apt install -y docker.io docker-compose-v2
sudo usermod -aG docker "$USER"      # log out and back in
./devenv/bootstrap.sh                 # Moodle 5.2 + PostgreSQL at
http://localhost:8001
./devenv/seed.sh                      # synthetic courses CSE1I0I, CSE2CWA, CSE1PES
./devenv/fixtures/reload_fixture.sh   # one rubric-graded assignment, 5 marked
students, 15 fills
# In python/.env: PG* settings, LJA_MOODLE_TABLE_PREFIX=m_ (the devenv prefix; hosted
Moodle is usually mdl_)
python -m lja.cli --source moodle     # uses ../data-
fixtures/criterion_silo_map_CSE1I0I.csv

```

Dev credentials are `admin / Devpass1!` and must never be reused anywhere. The read-only web-service token setup (dedicated `lja_service` user, custom service `lja_readonly`, four functions) is an eight-step procedure in the root README under "Configure the read-only Moodle Web Service"; `python moodle_probe.py` verifies it.

Restarting Moodle after Docker restarts (containers show Exited (255)):

```

source devenv/env.sh
cd "$MOODLE_DOCKER_WORKDIR"
bin/moodle-docker-compose up -d
bin/moodle-docker-wait-for-db

```

Screen recording.  *TO FILL*. A five-minute recording of steps 1–8 above on a fresh VM is planned for Sprint 6 (Anup's clean-rebuild task is the natural moment to capture it). Link it here.

4.4 Configuration reference

All configuration is environment variables, read in exactly one place: `python/lja/config.py`. `python/.env` is loaded automatically and is ignored. Every value below has a default; `.env.example` documents them.

Variable	Default	Purpose
LJA_LLM_PROVIDER	openai_compatible	anthropic or openai_compatible
ANTHROPIC_API_KEY	empty	Required only for anthropic
LJA_ANTHROPIC_MODEL	claude-opus-4-8	Claude model id
LJA_ANTHROPIC_EFFORT	empty (API default)	low...max. This model rejects temperature/top_p/top_k with a 400; effort and thinking are the knobs.
LJA_ANTHROPIC_THINKING	false	Adaptive thinking
LJA_OPENAI_BASE_URL	http://localhost:11434/v1	Ollama. LM Studio is :1234/v1. Note the /v1.
LJA_OPENAI_MODEL	qwen3-vl:30b	See §3.9 before choosing anything smaller
LJA_OPENAI_API_KEY	not-needed	
LJA_OPENAI_MAX_TOKENS	16000	Reasoning models need a large budget or they return empty content
LJA_OPENAI_TEMPERATURE	0.2	Classification task, keep it low
LJA_GAP_ABSOLUTE_FLOOR	50.0	Always a gap below this
LJA_GAP_ABSOLUTE_CEILING	75.0	Never a gap at or above this
LJA_GAP_RELATIVE_GAP_CUTOFF	-1.0	MAD units below the student's median → gap
LJA_GAP_RELATIVE_STRONG_CUTOFF	1.0	MAD units above → proficient
LJA_GAP_MIN_COMPETENCIES	4	Fewer → absolute fallback, recorded in the basis column
LJA_GAP_MIN_SPREAD	1.0	MAD below this = flat profile → fallback. Read ADR 0001 before changing.
LJA_GAP_FALLBACK_PROFICIENT	65.0	Proficient/developing split on the fallback path
LJA_SUBJECT_SEQUENCE	CSE100F, CSE2ALG, CSE3CAP	The intended order of subjects (§3.11). Where the owner's course map goes. An unlisted subject falls back to its year digit
LJA_TREND_STABLE_BAND	5.0	First-to-last difference, in points, inside which a trend is stable. Unratified (A-01)
PGHOST / PGPORT / PGDATABASE / PGUSER / PGPASSWORD	localhost / 5432 / moodle / lja_reader / empty	Moodle path
LJA_MOODLE_TABLE_PREFIX	mdl_	devenv uses m_
MOODLE_URL / MOODLE_TOKEN	—	moodle_probe.py only
LJA_DASHBOARD_EXCEL_PATH	../data-fixtures/CSE_results_150_students_3_Subjects.xlsx	Dashboard data
LJA_DASHBOARD_CLUSTERING_CACHE	output/silo_clustering.json	Dashboard cache
LJA_DASHBOARD_PLANS_DIR	output/plans	Where the dashboard looks for learning_plan_<id>.json (D7)
LJA_DASHBOARD_QUIZZES_DIR	output/quizzes	Where the dashboard looks for quiz_<id>.json (PR #52)
LJA_DASHBOARD_GENERATE	unset (off)	1 shows Generate buttons on the student page (D20); same as --allow-generate
LJA_QUIZ_CATALOGUE	../data-fixtures/subject_catalogue.yaml	Subject titles and handbook synopses for the quiz; skipped if missing (PR #52)
LJA_EMBED_MODEL	nomic-embed-text	Embedding model for the catalogue competency tagger, always via the OpenAI-compatible endpoint (ollama pull nomic-embed-text)
LJA_EXPORT_SALT	empty	Key for lja.export --anonymise. No default on purpose; empty refuses the run. Keep it out of git
LJA_RUN_MOODLE_INTEGRATION	unset	1 enables the one live-database test

4.5 Command reference

Run everything from `python/` with the `lja` environment active.

Command	What it does	Key flags	Exit codes
<code>python -m lja.cli [xlsx]</code>	Load → cluster (cached) → gate → gaps → CSVs	<code>--source excel moodle</code> , <code>--mapping</code> , <code>--clustering-cache</code> , <code>--review-file</code> , <code>--allow-unconfirmed</code> , <code>--refresh-clustering</code> , <code>--extra-instructions</code> , <code>--gaps-out</code> , <code>--clusters-out</code> , <code>threshold overrides</code>	0 ok; 2 gate blocked
<code>python -m lja.review</code>	Confirm/reject clusters	<code>--cluster <id or prefix></code> , <code>--state</code> , <code>--note</code> (required to reject), <code>--clustering-cache</code> , <code>--review-file</code>	
<code>python -m lja.plan [xlsx] STUxxxx</code>	Grounded learning plan for one student	<code>--source</code> , <code>--clustering-cache</code> , <code>--review-file</code> , <code>--out-dir</code> , <code>--max-attempts</code> , <code>--extra-instructions</code>	0 ok; 1 never grounded; 2 precondition (no cache, rejected cluster)
<code>python -m lja.strategy [xlsx] STUxxxx</code>	Grounded study strategy, one entry per gap	Same as <code>lja.plan</code>	0 ok or no gap; 1 never grounded; 2 precondition
<code>python -m lja.quiz [xlsx] STUxxxx (PR #52)</code>	Grounded practice quiz per gap, with a blind educator review	Same as <code>lja.plan</code> , plus <code>--catalogue</code> , <code>--items-per-gap</code> , <code>--format multiple_choice written mixed</code> , <code>--skip-educator-review</code>	0 ok or no gap; 1 never grounded; 2 precondition
<code>python -m lja.export [xlsx] --out DIR</code>	Three research CSVs and a manifest from the cached run; no LLM	<code>--source</code> , <code>--clustering-cache</code> , <code>--anonymise</code>	0 ok; 2 no cache or empty salt
<code>python -m lja.grounding_audit</code>	The IOLG-121 audit: plans for a fixed ten students with every attempt recorded, plus evidence pairs		
<code>python -m lja.dashboard</code>	Serve the dashboard	<code>--excel-path</code> , <code>--clustering-cache</code> , <code>--host</code> , <code>--port</code> , <code>--absolute-floor</code> , <code>--absolute-ceiling</code> , <code>--allow-generate (D20)</code> ; <code>LJA_DASHBOARD_PLANS_DIR</code> , <code>LJA_DASHBOARD_QUIZZES_DIR</code>	1 if cache missing
<code>python -m lja.data.synth_generator src.xlsx --add N --out f.xlsx</code>	Add synthetic students to the supplied workbook, with planted gaps	<code>--planted-gap-silos</code> , <code>--seed</code> , <code>--no-llm-feedback</code>	
<code>python -m lja.data.catalogue_generator catalogue.yaml --students N --out f.xlsx</code>	Generate a cohort with per-competency ability, programs and ground truth; optional Moodle fixtures	<code>--seed</code> , <code>--competency-sd</code> , <code>--planted-gap-*</code> , <code>--program-selection</code> , <code>--latent-share</code> , <code>--moodle-out</code>	
<code>python -m lja.data.catalogue_verify truth.json --gaps gap_report.csv</code>	Score a run against ground truth	<code>--clustering</code> , <code>--min-recall</code>	1 if below recall
<code>python -m lja.data.catalogue_draft catalogue.yaml --subject CODE "Title" YEAR ...</code>	Have the LLM draft synthetic subjects in the catalogue's structure	<code>--out</code>	
<code>python -m lja.data.handbook --year 2026 --prefix CSE ... -out raw.yaml</code>	Crawl real subjects and SILOs from the La Trobe handbook (1 request/s, disk-cached)	<code>--prefix</code>	
<code>python -m lja.data.competency_tagger raw.yaml --out tagged.yaml</code>	Group SILOs into competencies with embeddings and k-means; the LLM only labels	<code>--k</code> , <code>--traits</code>	
<code>python moodle_probe.py</code>	Web-services connectivity check	reads <code>MOODLE_URL</code> , <code>MOODLE_TOKEN</code>	
<code>python -m pytest -q</code>	Tests	<code>--cov=lja</code>	
<code>ruff check .</code>	Lint		

4.6 Verifying an installation

Check	Expected
<code>python -m pytest -q</code>	261 passed, 1 skipped on main at 2 Oct (335 on the PR #52 branch; counts will drift; zero failures is the criterion)
<code>ruff check .</code>	All checks passed!
<code>python -m lja.cli <xlsx></code> with a confirmed review	Prints a cluster table, then <code>N</code> gap rows, <code>M</code> persistent-gap students; <code>output/gap_report.csv</code> exists

Check	Expected
<code>python -m lja.dashboard</code> then <code>curl -s localhost:8000/ \ grep -c 'STU0'</code>	150
<code>curl -s localhost:8000/cohort/at-risk</code>	404 (deliberate, §3.7 D11)
<code>python -m lja.plan <xlsx> STU0003</code>	.json and .md in output/plans/, grounding summary printed
<code>python -m lja.strategy <xlsx> STU0003</code>	.json and .md in output/strategies/; the persistent entry names two subjects
<code>python -m lja.export <xlsx> --out output/export</code>	Four files; competencies.csv has 750 rows and students.csv 150 on the supplied workbook, with no LLM call
<code>curl -s localhost:8000/run \ grep -c LJA_GAP_</code>	7 (one row per threshold)

5. Ongoing maintenance

5.1 Repository map: where to change what

LearningJourneyAssistant/ ├─ README.md	Quick start, macOS guide, CI and branch-protection notes, team process
├─ requirements.txt	Pinned pip deps (what CI installs)
├─ .github/workflows/ci.yml	lint / test / security jobs
├─ python/ │ ├─ environment.yml	conda env "lja" (unpinned, conda-forge)
│ ├─ pyproject.toml	ruff config only; pytest rootdir anchor (run pytest from here)
│ ├─ .env.example	every setting, documented
│ └─ moodle_probe.py	Sprint 1 Web Services spike
│ └─ lja/ │ ├─ config.py	ALL env vars. Add a setting here and in .env.example, nowhere else
│ ├─ cli.py	pipeline command (347 lines)
│ ├─ plan.py	learning-plan command (173)
│ ├─ strategy.py	study-strategy command (120)
│ ├─ quiz.py	practice-quiz command (PR #52) (188)
│ ├─ export.py	research export, no LLM (265)
│ ├─ grounding_audit.py	the IOLG-121 ten-plan audit (282)
│ └─ review.py	staff gate: models, transitions, CLI (333)
│ └─ llm/ │ ├─ base.py	Protocol
│ ├─ factory.py	factory
│ └─ anthropic_client.py	anthropic client
├─ openai_compatible_client.py	openai_compatible_client.py · grounding.py · embeddings.py
├─ data/ │ └─ excel_loader.py	excel_loader.py · moodle_loader.py · sql.py · loading.py ·
├─ synth_generator.py	catalogue.py · catalogue_generator.py (585) · catalogue_verify.py
├─ catalogue_draft.py	catalogue_draft.py · moodle_emitters.py
├─ handbook.py	handbook.py (crawl) · competency_tagger.py (embeddings + k-means)
├─ silo_clustering.py	silo_clustering.py · gap_detection.py · gap_evidence.py ·
├─ trajectory.py (IOLG-106)	trajectory.py (IOLG-106) · learning_plan.py · study_strategy.py (379)
├─ silo_quality.py (532)	silo_quality.py (532) · quiz.py (732, PR #52)
├─ app.py (routes)	app.py (routes) · run_info.py · generate.py (IOLG-137 jobs) ·
├─ stats.py	stats.py · __main__.py · templates/ (21 files; _macros.html holds the tile and scrollbox macros)
├─ generate.js	static/{style.css, sort.js, scrollbox.js, chartzoom.js,
├─ tests/	30 files, 273 test functions, all offline;
├─ tests/validation/cases.csv	is the IOLG-110 worksheet
├─ output/	gitignored: caches, review file, CSVs, plans, strategies, quizzes,
├─ export	export
├─ sql/	moodle_attainment_extraction.sql (Queries 1-6), lja_reader DDL,
├─ README with the join gotchas	README with the join gotchas
├─ data-fixtures/	the supplied workbook, criterion-SILO map, subject_catalogue.yaml,
├─ .mbz sample	reference-run/ (cache, review file, plans; plus Sui Lung's review
├─ verdicts)	verdicts) · README-handbook-cohort.md
├─ handbook/	handbook/ is gitignored: La Trobe's content, regenerated in
├─ minutes	minutes
├─ devenv/	bootstrap.sh · seed.sh · env.sh · fixtures/ (rubric marking
├─ scripts)	scripts)
├─ docs/	docs/
├─ adr/	0001 relative gap detection · 0002 no-embedding clustering
├─ meetings/actions.md	the actions register (A-nn); read this before re-deciding anything
├─ sprint-plan.md, *.pdf	sprint plans (Rev 5 PDF is binding), pipeline and UML PDFs (Aug;
├─ regenerate)	regenerate)
├─ lecture_summaries/	W1-W10 course guidance
├─ Sprint 5 evidence:	validation-worksheet.md (IOLG-110) · grounding-audit.md (IOLG-121)
├─ security-evidence.md (IOLG-111)	security-evidence.md (IOLG-111)
├─ uat-checklist.md (IOLG-130)	uat-checklist.md (IOLG-130)
├─ risk-register.md (IOLG-125)	risk-register.md (IOLG-125) · cluster-review-sprint5.md (IOLG-
├─ 124/133) · export-schema.md	124/133) · export-schema.md + export-sample.ipynb (IOLG-120)
├─ sprints/sprint-5/	sprints/sprint-5/
├─ preflight and UAT transcripts	the raw evidence behind those: plan/evidence pairs, scanner logs,
├─ handover/	handover/
└─ traceability note	this document, the User Document and the learning-plan

Rule of thumb. A new setting goes in `config.py` and `.env.example`. A new data source goes in `lja/data/` and returns `LjaDataset`. A new LLM provider implements `LLMClient` and is wired in `factory.py`. A new generated artefact gets its own command and its own `ReferenceCheck` list. A new dashboard view is a route in `app.py`, a template, and a `TestClient` test; a new count is a `tile` and a list page that a test proves agree. Every one of these has an existing example to copy.

5.2 Routine operations

Task	How	Frequency
Re-run the pipeline on new results	Replace or point at the workbook (or Moodle), <code>python -m lja.cli ...</code> . The clustering cache is reused if it still covers every SILO; new SILOs trigger re-clustering and new clusters arrive as <code>pending</code> .	Each semester / assessment round
Re-cluster from scratch	<code>--refresh-clustering</code> . Existing review decisions survive for any cluster whose	When the model or prompt changes

Task	How	Frequency
	membership is unchanged (id is a hash of members).	
Review new clusters	<code>python -m lja.review</code> , then re-run the pipeline. Rejected clusters need a <code>--note</code> ; the note is printed as rework instructions.	After any re-clustering
Swap the LLM	Edit <code>.env</code> ; nothing else. Then re-cluster and re-review, because clustering output is model-dependent. Record the model in the ticket (A-29 asks for it in the cache too).	As governance allows
Regenerate the per-student artefacts	After a pipeline run, <code>lja.plan</code> , <code>lja.strategy</code> and (PR #52) <code>lja.quiz</code> for the students who need them; each is one or two LLM calls per student and nothing re-runs them for you. Point <code>LJA_DASHBOARD_PLANS_DIR</code> and <code>LJA_DASHBOARD_QUIZZES_DIR</code> at the results.	After any pipeline run
Export for the department	<code>python -m lja.export ... --out output/export [--anonymise]</code> ; keep <code>manifest.json</code> with the CSVs, it is what makes two exports comparable.	Each semester
Restart the dashboard	It has no reload; kill and rerun <code>python -m lja.dashboard</code> .	After any pipeline run
Rotate secrets	Replace <code>ANTHROPIC_API_KEY</code> / <code>MOODLE_TOKEN</code> / <code>PGPASSWORD</code> in <code>.env</code> ; revoke the old one at the provider. Never paste a token into a ticket or chat. <code>LJA_EXPORT_SALT</code> is different: rotating it breaks the link between old and new exports, so keep one salt per study and store it like a key.	On any suspected exposure; each semester
Clear generated state	<code>rm -rf python/output/</code> removes caches, CSVs and plans and the review file . Back up <code>*.review.json</code> first.	Rarely
Back up	<code>python/output/*.review.json</code> , <code>data-fixtures/criterion_silo_map/*.csv</code> , <code>.env</code> (encrypted). Everything else is regenerable or in git.	Before upgrades

5.3 Keeping the dependencies current

- `requirements.txt` is fully pinned and is what CI installs. `python/environment.yml` is unpinned and is what developers use. Keep them in agreement: after changing one, regenerate the other (`pip freeze` from a fresh env, or `conda env export --from-history`).
- Read the `pip-audit` output in the Security job of every CI run. It is non-blocking today. Triage the initial findings, then set `continue-on-error: false` in `ci.yml` so it blocks.
- Bump one family at a time and run the suite: SDKs (`anthropic`, `openai`) change structured-output APIs between minors, and the tests for both clients are regression tests against real bugs found live, so a red test after an SDK bump is signal, not noise.
- `ruff` is pinned to 0.16.4 on purpose. Bump it in its own PR, because a new rule can redder `main`.
- GitHub Actions: `actions/checkout@v4` and `setup-python@v5` will need bumping off Node 20 (A-19).
- Python 3.13: untested. The code uses nothing 3.12-specific, but `psycopg2` wheels and `pandas` 3 are the likely friction points.

5.4 CI and branch protection

Every push to `main` and every PR runs three independent jobs (§3.2). Branch protection on `main` (since 6 Sep 2026) requires one approving review from a non-author, all three checks green, the branch up to date, and applies to admins. It was verified by a PR with a deliberately failing test (PR #11) being refused.

Gates deliberately left soft, with the condition for tightening each:

Gate	Today	Tighten when
Coverage	Reported	Add <code>--cov-fail-under=80</code> once <code>cli.py</code> and <code>dashboard/_main_.py</code> have tests (they are at 0%; core modules are 98–100%)
<code>pip-audit</code>	<code>continue-on-error: true</code>	After the first triage
E501 line length	ignored	In one PR that reformats the 41 long lines and touches nothing else

To change protection rules you need repository admin; the settings and the reason for each are recorded in action A-03.

5.5 Extending the system

Add a data source. Write `lja/data/<source>_loader.py` returning `LjaDataset`; add a branch in `loading.load_dataset_for_source()` and a cache-path default in `clustering_cache_path()`; add `--source`

<name> to cli.py and plan.py. Copy test_moodle_loader.py for the test shape (fake cursor, one integration test behind an env var).

Add an LLM provider. Implement the three methods of LLMClient in lja/llm/<provider>_client.py. complete_structured must return a validated schema instance or raise; never return free text. Add the provider name to factory.get_llm_client() and the settings to config.py. Test with fake SDK objects, as both existing clients do.

Add a generated artefact. Follow learning_plan.py, or better study_strategy.py and quiz.py (PR #52), which are the two later copies of the same pattern: a Pydantic schema with extra="forbid"; a *Context dataclass whose known_* properties define the vocabulary; a list of ReferenceChecks including a regex scan of every prose field, filed per competency where the artefact is per gap; a generate loop that quotes grounding errors back and fails closed; its own python -m lja.<name> command built on build_plan_context() that respects the review gate and exits 0 without an LLM call when the student has nothing to act on. If the dashboard shows it, the page reads the JSON from a directory setting and never generates. The tender says quizzes are "most likely to look plausible while quietly not being grounded", which is why this discipline is not optional, and \$3.10 records the limit grounding cannot cross.

Add a generated artefact to the Generate button. One entry in ARTEFACTS in dashboard/generate.py (module, output file name, label) and a generate_box call in student.html beside the section that renders it; the job runner, routes and polling script are shared. Study strategies are the obvious candidate once the page shows them.

Add a dashboard view. A route in app.py returning a template that extends base.html (so the review banner appears); compute from dataset, gaps and clustering passed to create_app, never from the LLM; a TestClient test in test_dashboard.py. Every count is a tile that links to a list page, membership is decided in app.py and a test in test_dashboard_lists.py proves the list holds what the tile counted. Long tables go in the scrollbar macro; charts come before lists and plot the same rows. The subject, assessment and /run pages in PR #43 are the worked examples. Cohorts are registered in _COHORTS (five today: all, persistent-gap, priority-1, -2, -3).

Change a threshold. Change the default in config.py and the comment in .env.example, and update ADR 0001's status from "Proposed" only when the owner has ratified the number on data with ground truth (catalogue_verify gives recall against planted gaps). test_thresholds_are_configuration_not_constants will catch a hardcoded literal.

Scale clustering past ~50 SILOs. See \$3.9 item 4 and ADR 0002's review triggers. The catalogue generator produces the test data, and the handbook crawl supplies real SILOs at scale; catalogue_verify scores pairwise precision/recall of any clustering against the catalogue's competency tags.

5.6 Troubleshooting

Symptom	Cause	Fix
lja.cli exits 2: "clusters pending review"	Staff gate	python -m lja.review, or --allow-unconfirmed for an exploratory run
lja.cli exits 2 and prints rework instructions	A cluster is rejected	Fix the prompt/model (--extra-instructions, --refresh-clustering), or reset the cluster to pending
GroundingError: missing from any cluster after 3 attempts	Model too small, or SILO count too high	Use qwen3-vl:30b or Anthropic; see \$3.9
Empty LLM content, finish_reason=length	Reasoning model spent the budget thinking	Raise LJA_OPENAI_MAX_TOKENS; confirm the server honours response_format
400 from Anthropic mentioning temperature	claude-opus-4-8 rejects sampling params	Use LJA_ANTHROPIC_EFFORT / LJA_ANTHROPIC_THINKING
"JSON error" after a very long wait	Server hung; timeout misreported (A-30)	Check the Ollama/LM Studio process; there is no LJA_OPENAI_TIMEOUT yet
lja.plan, lja.strategy or lja.export exits 2: No clustering cache at output/silo_clustering.json right after the quick start	The quick start read the reference cache but wrote nothing under output/ (UAT-01)	Pass the same --clustering-cache ../data-fixtures/reference-run/silo_clustering.json to the artefact command
lja.export --anonymise exits 2	LJA_EXPORT_SALT is empty	Set a salt in .env; never commit it
lja.strategy or lja.quiz prints one line and writes nothing	The student has no isolated or persistent gap	Nothing to fix; exit 0 is correct
Quiz written without educator notes	The review pass never grounded; it is a warning	Re-run, or review the questions by hand; --skip-educator-review to skip the call deliberately
No Generate button on the student page; POST .../generate/plan returns 403	Generation is off by default (D20)	Start with --allow-generate; the Provenance page confirms it is on
Generate shows "Failed" with the command's output	The CLI exited non-zero: no model reachable, the gate blocked (exit 2), or no attempt grounded (exit 1)	Read the output lines; the previous file, if any, is untouched
Dashboard exits 1 at start	No clustering cache	Run lja.cli first, or point --clustering-cache at the reference run (PR #22)
Dashboard charts blank	No internet for the CDN	Vendor Chart.js and d3 (A-20) or connect
Banner says a cluster was rejected but not which	Known gap (UAT-02)	Run lja.cli; the CLI names the rejected cluster and prints the rework note

Symptom	Cause	Fix
KeyError in compute_gaps	A SILO in the dataset is in no cluster	Cache is stale for this dataset; <code>--refresh-clustering</code>
ValueError: unmapped criterion on <code>--source moodle</code>	<code>criterion_silo_map</code> CSV lacks a row	Add the mapping row; it is required, not best-effort
Query 2 returns 0 rows	Wrong table prefix, or <code>grading_instances.status != 1</code>	Set <code>LJA_M00DLE_TABLE_PREFIX</code> (m_ in devenv); check <code>reload_fixture.sh</code> ran
<code>pg_config</code> executable not found on pip install	psycpg2 builds from source	<code>apt install libpq-dev</code>
Moodle containers Exited (255)	Docker restarted	\$4.3 restart block
<code>pytest</code> collects nothing / wrong rootdir	Run from the wrong directory	<code>cd python</code> first
Lint red on <code>main</code> after merging	I001 import order	<code>ruff check --fix</code> in its own commit

5.7 Technical debt register

Things a maintainer should know are known. Each has an owner in the actions register or a ticket where one exists.

#	Item	Where	Ref
1	Chart.js from an unpinned CDN	<code>dashboard/templates/base.html</code>	A-20
2	No <code>LJA_OPENAI_TIMEOUT</code> ; timeouts misreported as JSON errors	<code>openai_compatible_client.py</code>	A-30
3	Cache does not record which model produced it	<code>cli.py</code>	A-29
4	<code>pip-audit</code> non-blocking; E501 ignored	<code>ci.yml</code> , <code>pyproject.toml</code>	A-12, A-13
5	Performance-band cut points (50/60/70/80) duplicated in three modules	<code>moodle_loader.py</code> , <code>catalogue_generator.py</code> , <code>synth_generator.py</code>	—
6	Feedback-band cut points (50/65/80) in two places	<code>synth_generator.py</code> , <code>moodle_emitters.py</code>	—
7	Trend band reasoned, not measured. <i>Now <code>LJA_TREND_STABLE_BAND</code>, configuration shown on <code>/run</code> (IOLG-106); the value is still unratified</i>	<code>config.py</code>	A-01
8	Year level parsed from the subject code (CSE100F → 1). <i>Now the documented fallback behind <code>LJA_SUBJECT_SEQUENCE</code>, and each card says when it was used (IOLG-106); the remaining debt is that no course map has been supplied</i>	<code>trajectory.py</code>	FR-1.7
9	<code>lja.review --clustering-cache</code> not source-aware	<code>review.py</code>	—
10	Moodle loader hardcodes weight 1.0, no early/hurdle	<code>moodle_loader.py</code>	—
11	SQL Query 6 carries legacy 50/65 thresholds, annotated divergent	<code>sql/</code>	—
12	Competency-framework CSV emitter writes <code>scaleid 0</code>	<code>moodle_emitters.py</code>	—
13	Docstrings say <code>psycpg.connect</code> ; code uses <code>psycpg2</code>	<code>config.py</code> , <code>sql.py</code>	—
14	Root <code>environment.yml</code> is a stale machine-specific export; <code>python/environment.yml</code> is the real one	repo root	delete it
15	Untracked report files in the repo root (*.html, sprint reports, temp)	repo root	move under <code>docs/sprints/</code> or delete before the handover zip
16	Jira epics triplicated (IOLG-63–77)	Jira	A-34
17	Review banner reports a count, not the rejected cluster's name	<code>dashboard/templates/base.html</code>	UAT-02
18	<code>lja.export</code> has no staff-gate check; plans and strategies warn on pending rather than block	<code>export.py</code> , <code>plan.py</code> , <code>strategy.py</code>	compliance SE-5

#	Item	Where	Ref
19	Quick start's cache path differs from the bare artefact commands	root README.md, python/README.md	UAT-01, IOLG-114
20	Quiz answer key is unverifiable; the blind reviewer defaults to the same model	model/quiz.py (PR #52)	§3.10
21	d3 and d3-cloud also from the CDN, unpinned to a minor	dashboard/templates/silos.html	A-20
22	The architecture and use-case diagrams predate the Sprint 5 commands and pages	docs/handover/diagrams/01-*.mmd, 02-*.mmd	this document
23	/run reconstructs the command from argv; a dashboard embedded by another caller gets no command section	dashboard/run_info.py	—

5.8 The Moodle production path: what an operator must know

Stated plainly because the owner asked for it. LJA reads rubric fills **directly from the Moodle database**, not through the Moodle API, because the API has no function that returns them.

- **Access.** Create the `lja_reader` role with the DDL in `sql/README.md` (`LOGIN`, `CONNECT`, `USAGE`, `SELECT ON ALL TABLES`, default privileges for future tables). Never connect as the Moodle application user. Prefer a read replica.
- **Never write.** Grade aggregation, event triggers and cache invalidation live in Moodle's PHP. A direct `UPDATE` silently desynchronises the gradebook. Nothing in LJA writes, and IOLG-111's checksum test (`pg_dump --data-only | md5sum` before and after a run) is how to prove that on your instance.
- **Schema coupling.** Query 2 joins `gradingform_rubric_fillings` → `grading_instances` (`status = 1` only; `itemid` is *not* a user id, it joins to `assign_grades.id`) → `assign_grades` → `assign` → `course` → `user`, plus the rubric criteria and levels tables. After any Moodle major upgrade, run `python -m pytest -q` with `LJA_RUN_MOODLE_INTEGRATION=1` against the devenv (expects 15 fills for the fixture) before trusting production output.
- **Table prefix.** `mdl_` on most hosted instances, `m_` in the devenv. It is `LJA_MOODLE_TABLE_PREFIX`.
- **Mapping.** One `criterion_silo_map_<SUBJECT>.csv` per subject, staff-editable, required. Moodle's Outcomes and Competency tables are not used because they were empty for the subjects examined.
- **Web Services** are used only for discovery and the connectivity probe, with a four-function token on a dedicated service account. Do not generate an admin token.

5.9 Documentation that is known to be stale

IOLG-114 (PR #27) landed the handover set, but the four README items below were checked again on 2 Oct and are still present. Fix them in the next README pass, together with UAT-01:

- `python/README.md` "Not yet written" still lists the staff confirmation workflow; `review.py` implements it and `cli.py` enforces it.
- `python/README.md` says plans do not consult review states; `plan.py` does since IOLG-108.
- `synth_generator.py` docstring mentions a `--verify` flag that does not exist.
- `devenv/env.sh` says "all six of us"; the team is five.
- `docs/LJA – Data & Algorithm Pipeline.pdf` and the UML PDF are dated 11–12 Aug and show the LLM layer and dashboard as "planned". The diagrams in §3 of this document supersede them.

6. Testing

6.1 Strategy

Tests were written before or with the code, derived from the acceptance criteria on each sprint story (the Sprint 3 runbook rule). Every test is offline: no test calls a real LLM or a real database unless

`LJA_RUN_MOODLE_INTEGRATION=1` is set. LLM access is replaced by hand-rolled fake clients that return canned Pydantic objects, or by fake SDK objects patched onto the client. This keeps the suite at under two seconds and free to run, and means CI proves the *code*, while model quality is evaluated separately (§6.5).

Level	What	Where	Automated
Unit	Every module in <code>lja/</code> except <code>config.py</code> , <code>factory.py</code> , <code>catalogue_draft.py</code> and the two <code>__main__</code> entry points	<code>python/tests/test_<module>.py</code>	Yes, CI
Component / CLI	<code>plan.py</code> , <code>strategy.py</code> and (PR #52) <code>quiz.py</code> gate behaviour with the pipeline monkeypatched; <code>export.py</code> end to end on an in-memory dataset	<code>test_plan_cli.py</code> , <code>test_strategy_cli.py</code> , <code>test_quiz_cli.py</code> , <code>test_export.py</code>	Yes, CI
Integration	Dashboard routes via FastAPI <code>TestClient</code> over in-memory data, including every list page against its tile; generator → workbook → loader → <code>compute_gaps</code> end to end; SQL Query 2 against a live Moodle	<code>test_dashboard.py</code> (38), <code>test_dashboard_priority.py</code> (11), <code>test_dashboard_lists.py</code> (13), <code>test_catalogue_generator.py</code> , <code>test_moodle_loader.py::test_integration_loads_seeded_subject</code>	Yes; live DB test skipped unless enabled
Regression	Bugs found against live models: empty content on <code>finish_reason=length</code> , <code>json_schema</code> rejected by a server, a thinking block arriving before the text block, Average Total leaking into subject totals	client and loader tests	Yes, CI
Static	ruff E/F/I; gitleaks full history; pip-audit	CI	Yes
Acceptance (UAT)	S1–S6, D1–D10 walked with the project owner from a clean clone	<code>docs/uat-checklist.md</code> : S1–S6 run locally on 28 Sep as preliminary evidence (§6.5); live review 4/5 Oct	No, manual
Validation	Seven hand-worked gap-detection cases, now also a pinned test; ten plans audited by machine and pre-reviewed; six security evidence results; clean-machine rebuild preflight	<code>test_validation_cases.py</code> over <code>tests/validation/cases.csv</code> ; <code>docs/grounding-audit.md</code> ; <code>docs/security-evidence.md</code> ; <code>docs/rebuild-check.md</code>	Cases yes; the rest manual (§6.5)

Suite size on 2 Oct 2026: 30 files, 273 test functions, 261 pass, 1 skipped (live Moodle), 2.9 s on `main`; 335 pass on the PR #52 branch. At 0.1 (26 Sep) it was 195 passing. Coverage on the core modules (`gap_detection`, `silo_clustering`, `grounding`, `learning_plan`, `review`) is 98–100%; the headline figure is lower because `cli.py` and `dashboard/__main__.py` have no direct tests.

6.2 User-story test table

Story	Unit tests	Integration
E1 Semantic clustering	<code>test_silo_clustering.py</code> (7): complete coverage accepted; missing, duplicated and unknown SILO rejected; retry recovers from a bad first attempt; gives up after max attempts. <code>test_grounding.py</code> (12).	Live runs against <code>gemma4</code> , <code>qwen3-vl:30b</code> , <code>qwen</code>
E2 Staff gate	<code>test_review.py</code> (8): pending → confirmed, reject requires note, settled decisions cannot flip directly, state survives reload, gate reports pending and rejected. <code>test_plan_cli.py</code> (3 of 4). <code>test_cli.py</code> (3): stale cache detection.	<code>test_dashboard.py</code> banner tests (3)
E3 Relative gap detection	<code>test_gap_detection.py</code> (14): median/unscaled MAD; relative gap flagged although it is a pass; uniformly weak still gets gaps (floor); uniformly strong gets none (ceiling); flat profile and too-few-competencies fall back and say so; persistent/isolated split; thresholds are configuration. <code>test_gap_evidence.py</code> (10). <code>test_validation_cases.py</code> : the seven IOLG-110 profiles (28 rows) with hand-worked expected labels, thresholds pinned.	<code>test_catalogue_generator.py::test_truth</code> <code>catalogue_verify</code> planted-gap recall (32/33 on
E4 Grounded plan	<code>test_learning_plan.py</code> (17): invented SILO, subject, assessment, competency, strength and wrong student all rejected; codes invented inside prose caught; every problem reported in one error; retry with errors quoted; build fails when no attempt grounds.	First live run recorded (35 s, 3.8k in / 0.4k out); ten-attempt, 190 s, \$0.62

Story	Unit tests	Integration
S1 Pipeline on workbook	test_excel_loader.py (7/8), test_loading.py (7)	test_catalogue_generator.py round trip
S2 Pipeline on Moodle	test_moodle_loader.py (9), test_sql.py (6): prefix substitution, Query 2 is one statement	test_integration_loads_seeded_subject (
S3 Learning plan	as E4; test_plan_cli.py::test_moodle_source_resolves_the_moodle_cache_default	—
S4 Export	test_export.py (5): three CSVs and manifest, pseudonyms stable under one salt and different under another, empty salt refused, in_plan from the plan JSON	UAT S4 local check: 150/750/5 rows, notebook ren
S5 Review changes the banner	test_dashboard.py: pending warning, rejected warning, hidden when confirmed	—
S6 Tests pass	the suite	CI on every PR
D1 Cohort list	test_index_lists_every_student, ... counts_only_students_with_a_persistent_gap, ... marks_at_risk_students_link..., ...table_is_marked_sortable...	TestClient
D2 Drill-down with reason	...cohort_page_contains_only_its_members, ... states_what_put_students_in_it, ...tile_count_and_row_count_agree, ... unknown_cohort_is_404..., ...at_risk_cohort_is_not_registered_yet	TestClient
D3 Understanding chart	...student_detail_shows_its_gaps, ...worst_classification_renders_first; test_dashboard_stats.py (8)	TestClient
S7 Study strategy	test_study_strategy.py (19): one entry per gap, none for a strength, per-competency grounding of subjects, assessments and SILOs, persistent needs two subjects and interleaving or spacing, isolated needs feedback review, closed technique list; test_strategy_cli.py (6)	Three live runs recorded, all first attempt
S8 Practice quiz	On the PR #52 branch: test_quiz.py and test_quiz_cli.py, one test per grounding and shape rule, the written-task rule, the educator review's checks, the gate and the fail-closed exit; test_dashboard.py renders a quiz and its empty state	Two live runs recorded (\$3.10)
D4 Strengths	...lists_proficient_competencies_under_strengths, ... ordered_strongest_relative_position_first, ... index_strength_count_matches_proficient_competencies	TestClient
D5 Gaps with evidence	...shows_per_subject_evidence_and_trend, ... flags_future_subjects_for_an_at_risk_gap, ...honest_empty_state..., ... never_flags_future_subjects_for_a_non_gap	TestClient
D6 Progress	...shows_progress_across_subjects_in_year_order, ... marks_single_subject_competency_as_insufficient, ... progress_empty_state_without_evidence	TestClient
D13 Generate from the page	test_dashboard_generate.py (8), fake runner: no button and the real workbook path when off, 403; button and labels; the command is the real CLI invocation with this run's inputs; a job runs and the page then shows the plan and Regenerate; a failure keeps the old file; a second press is 409; unknown artefact or student is 404; Provenance states on/off	TestClient
D12 Trajectory	test_trajectory.py (17): parsing, fallback, boundaries, skipped subject, basis values; test_dashboard.py: ... subject_chain_in_sequence_order_with_the_ahead_subject_marked, ... order_came_from_the_year_digit_when_nothing_is_declared, ... only_offered_for_a_gap_not_a_strength, ... run_page_shows_the_declared_sequence..., ...glossary_defines_trajectory	TestClient
D7 Next actions	...renders_generated_learning_plan, ... shows_plan_empty_state_when_file_is_missing, ... without_plans_directory_is_safe	TestClient over a plan JSON in tmp_path
D8 Student picker	...header_picker_lists_every_student_on_every_page	TestClient
D11 Priority groups, /run, lists	test_dashboard_priority.py (11): group membership from the classifier's outputs, ordering by lowest flagged mark, /run thresholds and provenance; test_dashboard_lists.py (13): each list equals its tile	TestClient
D9 Unreviewed-AI banner	as S5; on the PR #25 branch the banner tests also assert it links to /clusters	—
D10 Traceability	...states_what_put_students_in_it; classification_basis asserted in test_gap_detection.py	—

6.3 Running the tests

```

cd python
python -m pytest -q                                     # whole suite, offline
python -m pytest -q --cov=lja --cov-report=term-missing
LJA_RUN_MOODLE_INTEGRATION=1 python -m pytest -q tests/test_moodle_loader.py # needs
devenv Moodle + fixture
ruff check .

```

pytest must run from python/ because pyproject.toml anchors the rootdir there. There is no conftest.py, no custom markers and no pytest config section; the suite is deliberately plain.

6.4 Continuous integration

See §3.2 and §5.4. The security job checks out full history so a secret that was committed and later “removed” is still found; a full-history scan was confirmed clean on 24 Aug 2026 and the job’s value is that it stays that way. Coverage XML is uploaded as an artifact on every run, including failed ones.

6.5 Manual validation in Sprint 5: what was done and what is still open

Status at 2 Oct 2026. All evidence files are on main.

Activity	Ticket	Owner	Evidence	Result
Seven hand-worked gap-detection cases against the engine	IOLG-110	Anup	docs/validation-worksheet.md; tests/validation/cases.csv; test_validation_cases.py	Done 28 Sep. 28 rows across seven profiles (the brief said 30; the profiles add to 28), expected labels worked by hand with the arithmetic shown, and pinned as a test so a local .env cannot change the answers. All pass.
Ten learning plans read for invented claims and tone	IOLG-121	Istiaque / Anup	docs/grounding-audit.md; docs/sprints/sprint-5/grounding-audit/ (plans, evidence pairs, machine-audit.csv, assisted-review.md)	Machine audit done 28 Sep: ten fixed students, claude-opus-4-8, 10/10 grounded on the first attempt, no retries, 45.5k tokens in, 15.8k out, 190 s, \$0.62. Assisted prose pre-review found concerns in eight plans (G01–G10): feedback attributed to the wrong assessment, a summary counting two persistent gaps where there is one, claims of progress or of missing work the evidence does not contain. Human sign-off pending (the review table’s cells are still Pending).
Six security evidence results	IOLG-111	Anup / Ayesha	docs/security-evidence.md; docs/sprints/sprint-5/security-evidence/ci-scanners-2026-09-28.txt	All six observed 28 Sep on the dev instance (PostgreSQL 17.11, Moodle 5.2.2, prefix m_): role and grants created; UPDATE m_user denied; 15 fills readable; data-only dump checksum identical before and after a run; probe token authorises 429 functions against 4 needed (not least-privilege; the production path does not use it); gitleaks 67 commits, no leaks; pip-audit no known vulnerabilities. Review by Anup and Ayesha pending before Done.
Tender compliance checklist migrated with evidence	IOLG-117	Anup	docs/compliance-checklist.md	Done 28 Sep: all 40 original items under the eight headings; snapshot 9 Done, 30 Open, 1 Deferred. The

Activity	Ticket	Owner	Evidence	Result
				Open items are mostly missing historical proof (before-first-push scans, the five-PR split), not missing controls.
Staff review of the AI cluster labels	IOLG-124, IOLG-133	Sui Lung / Istiaque	docs/cluster-review-sprint5.md; data-fixtures/reference-run/silo_clustering.review.sui-lung.json	Done 26 Sep, landed 2 Oct. Of the five reference-run clusters: four Agree, one Disagree (Object-Oriented Design and Implementation). Both SILOs the AI flagged as vague were agreed vague. The verdicts are in a separate review file so the all-confirmed reference run keeps working offline.
Project risk register re-scored	IOLG-125	Sui Lung	docs/risk-register.md	Done 27 Sep, row 1 refreshed 2 Oct with the IOLG-111 evidence. Twelve rows; the three highest residuals are Definition of Done not applied uniformly (12), thresholds unratified and over-flagging (16), and clustering not scaling past one call (16).
Clean-machine rebuild by a non-author, script-logged	IOLG-118	Anup	docs/rebuild-check.md; docs/sprints/sprint-5/rebuild/local-preflight-2026-09-28.txt	Not performed. The available environment was an assisted Mac workspace, not a fresh Ubuntu Codespace, and no log was fabricated. Four preflight findings recorded (\$4.3). The Jira ticket is In Review, which overstates this.
UAT checklist S1–S6 (system rows)	IOLG-130, IOLG-129	Anup / Ayesha	docs/uat-checklist.md; docs/sprints/sprint-5/uat/	Preliminary local run 28 Sep , live tester pending: S1 pass (750 rows, byte-identical to the reference report); S2 blocked (no Docker); S3 and S4 pass with the explicit cache flag, bare commands exit 2 (UAT-01); S5 fails the naming criterion (UAT-02); S6 pass (258 at the time). The combined checklist with Sui Lung's D1–D10 rows is to be landed for the review.
UAT D1–D10 walked with the owner	IOLG-127	Sui Lung	docs/uat-checklist.md (combined)	△ Live review 4/5 Oct.

7. Usability trial results

△ No usability trial with the project owner has been run yet. The trial is scheduled as part of the Sprint 5 review on 4/5 October 2026. This section sets out who will test, what they will do, what is already known from owner feedback and from the team's own preliminary runs, and the table to fill. It is framed as the project owner's roadmap because that is what the results are for.

7.1 Participants and method

Participant	Role in the trial	Tasks
Dr Scott Mann, project owner	Primary evaluator, acting as subject coordinator	D1, D2, D5, D9, D10; review of cluster labels (R3); comment on SILO quality views
Anup Tumbalam Gooty	Non-author operator, from a clean clone	S1–S6; records blockers on IOLG-114
Sui Lung Tang	Facilitator, records results	Walks D1–D10; records Result / Tester / Notes
Team members acting as students	Secondary	D3, D4, D8

Method: think-aloud walkthrough of each story from the running system, not slides. Each row is scored pass / pass with friction / blocked, with a note. Tender success criterion: "project-owner and team UAT completes all critical student tasks without a blocker and records improvement actions"; performance target "normal local dashboard interactions respond within approximately two seconds, excluding external LLM generation time".

7.2 What the project owner has already told us (11 Aug 2026 and since)

These shaped the build and should be re-tested in the trial:

- Semantic, not keyword, matching is the whole point. His worked example links CSE100F SILO2 to CSE2ALG SILO2/3. The 30B model found exactly that link ("Data Structures Knowledge and Application", six students). *Test: does he agree with the labels?*
- "Gap in first year, consequence in third year" is the narrative the dashboard must tell. *Test: D5's per-subject evidence and future-subject list.*
- There is no institutional at-risk number ("I'm not sure how we determined at risk"). *Test: is the absence of an at-risk cohort acceptable, and are the classification bases understandable?*
- The primary signal is variability within a discipline, not raw failure. *Test: E3 on his reading of a few students.*
- The power comes at scale, 30-plus subjects. *Test: show the 52-SILO failure and the catalogue generator; ask which of the scaling options he prefers.*
- Assessment weighting was deprioritised. *Test: confirm D13's approximation is acceptable.*
- DevSecOps is a differentiator. *Test: walk the six security evidence items.*

7.3 Results (△ TO FILL at the review)

The team's own preliminary results from 28 Sep are in the S rows so the review starts from them rather than rediscovers them; the owner's verdicts replace them. Two findings are already logged: **UAT-01** (bare artefact commands exit 2 after the quick start; documentation, routed to IOLG-114) and **UAT-02** (the banner does not name the rejected cluster; S5 fails on the dashboard path). Sui Lung's cluster-label review (R3) is complete and summarised in §6.5.

ID	Story	Result	Tester	Time	What they found	What they recommended
S1	Pipeline on workbook	pass (preliminary)	Anup, local, 28 Sep		750 gap rows, byte-identical to the reference report	
S2	Pipeline on Moodle	blocked (preliminary)	Anup, local, 28 Sep		No Docker daemon; live test skipped	Run at the review from the devenv
S3	Learning plan	pass with friction (preliminary)	Anup, local, 28 Sep		Bare command exits 2 (UAT-01); passes with the cache flag; no new generation without a provider	Carry the cache flag through the docs
S4	Export	pass with friction (preliminary)	Anup, local, 28 Sep		Same UAT-01; 150/750/5 rows; notebook renders	
S5	Review changes banner	fail (preliminary)	Anup, local, 28 Sep		Banner shows a count, not the cluster name (UAT-02); CLI names it	Amend the banner, or revise the criterion
S6	Tests pass	pass (preliminary)	Anup, local, 28 Sep		258 passed, 1 skipped at the time	
D1	Cohort list					

ID	Story	Result	Tester	Time	What they found	What they recommended
D2	Cohort drill-down					
D3	Understanding chart					
D4	Strengths					
D5	Gaps with evidence					
D6	Progress					
D7	Next actions					
D8	Student picker					
D9	Unreviewed-AI banner					
D10	Traceability					
R3	Cluster label review (per cluster, owner's comment)	4 Agree, 1 Disagree (team review)	Sui Lung, 26 Sep		OO Design cluster disagreed; both vague flags agreed	Owner to confirm or overrule at the review

7.4 Recommendations already on the table (team-sourced, pre-trial)

Recorded here so the trial can confirm or reject them rather than rediscover them:

- Viewer-adjustable histogram bins (A-18) and gap thresholds with a re-evaluate control (A-27), with an explicit "you are exploring sensitivity, not setting policy" guard.
- A competency lens: a `/competencies` index, competency tree, assessment leverage and the students per competency (`LJA_WP_competency-lens.md`).
- Name the rejected cluster in the review banner (UAT-02).
- A confirmation UI for the staff gate instead of the CLI.
- Whether staff want the practice quiz at all, and if so which format policy (`mixed`, `written`, `multiple_choice`) and which reviewer model (§3.10).
- Which relative-gap cutoff to ratify, now that the 3,000-student sensitivity table exists (§3.9 item 2), and whether the priority groups are the right way to present the result.

7.5 Roadmap for the project owner (*Δ to be finalised from 7.3*)

To be written as: what to keep as is, what to fix before any wider use, what to build next, and what to decide (thresholds, at-risk rule, provider governance, real de-identified data, handbook consent).

8. Future work

This chapter records what was descope, what was deferred, and the late suggestions, so the next team starts from decisions rather than from scratch. The pre-agreed descope order was quizzes → study strategies → learning plans, with the extraction → clustering → gap-detection spine non-negotiable. The spine was delivered.

8.1 Descoped

- **Adaptive quiz generation (R8), as specified.** Descoped by plan as the first candidate: highest complexity-to-value, and “most likely to look plausible while quietly not being grounded”. A thin slice was built late in Sprint 5 (PR #52, §3.10) following the learning-plan pattern, with every question citing the SILO and assessment it targets. What remains descope is the part the tender was worried about: the grounding checks cannot include the answer key, because nothing in the input can verify it. The slice is a staff tool until that changes.
- **Study-strategy recommendations (IOLG-123).** Built (PR #26, merged 28 Sep). No longer descope.
- **Feedback attribution layer and evidence panel.** Not built because 45 of the workbook’s feedback comments are templated strings. Needs the sanitised bespoke feedback extract the owner offered, and a re-identification review before it is used.
- **Threshold sensitivity chart.** Dropped from Sprint 5 as a chart; the numbers now exist as a table (§3.9 item 2, three cutoffs on 3,000 students) and `/run` bins every flagged mark in 5-point steps. A chart over `LJA_GAP_RELATIVE_GAP_CUTOFF` against planted-gap recall is still a two-hour task.

8.2 Deferred, with the reason

- **Embedding pre-pass for the pipeline’s clustering.** Deviation recorded in ADR 0002, and the 52-SILO failure hit the ADR’s own trigger. The tagger and embedding client are now on `main` (PR #29, D18) and have grouped 1,436 real SILOs in 64 seconds, but they feed the catalogue’s ground truth, not `cluster_silos()`. Wiring them into the pipeline, with the staff gate unchanged, is the next clustering work package.
- **Handbook crawl of real La Trobe SILOs.** Code on `main` (PR #29); the crawled data (`data-fixtures/handbook/`) stays gitignored pending the owner’s ruling on committing it. It regenerates in minutes from the sitemap. The 100-subject, 5,000-student cohort in `data-fixtures/README-handbook-cohort.md` is built from it.
- **Multi-subject Moodle extraction.** The loader works for one seeded subject. Generalising it is mostly fixture work (the catalogue generator’s `--moodle-out` emits per-subject frameworks and a rubric-marking JSON) plus removing the hardcoded assessment weight.
- **Trajectory model (IOLG-106).** Built on its own branch (§3.11): declared sequence, subject chain on the gap card, configurable band, basis label. What remains, in order: filter “ahead” to the student’s own program (the Program column exists in the workbook and the catalogue; the loader drops it), which is what makes the chain readable on a large cohort (§3.11); the owner’s course map to put in `LJA_SUBJECT_SEQUENCE` (FR-1.7, still a decision he holds); Moodle enrolments as the source of “ahead” on the production path; and the gap co-occurrence graph the ticket’s epic once mentioned, which was never specified.
- **Quiz answer-key verification.** Either real course material as a source to check against, or an independent reviewer model with rejection on disagreement (§3.10). Also: strategies are not yet shown on the dashboard.
- **Dashboard over the Moodle source.** The dashboard still takes `--excel-path` only; the loaders are shared, so this is plumbing in `dashboard/__main__.py`.

8.3 Late suggestions and engineering follow-ups

- Vendor and pin Chart.js and d3 (A-20). Add `LJA_OPENAI_TIMEOUT` (A-30). Record provider and model in the clustering cache (A-29). Add `reviewed_by/reviewed_at` to the review file (§3.8).
- Name the rejected cluster in the banner (UAT-02). Carry the cache flag through the quick start (UAT-01). Give `lja.export` a staff-gate check (SE-5).
- Resolve the ten prose findings of the grounding pre-review in the plan prompt (attribute feedback to one assessment; never claim progress or missing work the evidence does not show), then regenerate and have Anup sign off.
- Authentication and a per-role view (student sees self; coordinator sees cohort). Today there is none, which is fine for `127.0.0.1` and synthetic data and for nothing else.
- Pseudonymise at ingest (owner FR-1.5) before any real data touches a cloud provider.
- A confirmation UI for the staff gate; today it is a CLI.
- IRT or hierarchical simulation for synthetic students (A-28: `py-irt`, `girth`, PyMC) so threshold ratification rests on data with known latent ability.
- Chunked or hierarchical clustering with a merge pass, so the pipeline holds at 30-plus subjects.
- Reconcile SQL Query 6’s legacy thresholds with the Python engine, or delete Query 6.
- Tighten CI: `pip-audit` blocking, coverage gate, E501.
- A Moodle plugin exposing rubric fills as a web service (tender Option 3), which would remove the database credential from the production path entirely.

8.4 Production outlook (from the tender)

Roughly \$200 per month for a small managed instance; a maintenance allowance of about 40 hours per semester; LLM cost about \$0.02 for 150 students × 3 subjects on the Anthropic path, so about \$0.10 per 1,000 students per semester. The dominant cost of running LJA is people reviewing clusters and ratifying thresholds, not compute.

8.5 Decisions the project owner still holds

At-risk rule (or none); ratified gap thresholds, now with a sensitivity table to choose from; LLM provider governance; availability of real de-identified attainment data; course maps for the CS, IT and Cyber degrees; the institutional competency framework, if any; consent to commit crawled handbook content; the export field set (asked 23 Sep, still open); whether students may ever see a quiz before staff review, and which format policy; the licence for the repository.

Appendix A. Glossary

Term	Meaning
SILO	Subject Intended Learning Outcome. Numbered locally per subject: CSE100F:SILO1 and CSE2ALG:SILO1 are unrelated.
Competency	A cross-subject grouping of SILOs that describe the same underlying skill, produced by clustering and labelled by the LLM.
Cluster	The LLM's proposed competency: a label, a rationale and member SILOs.
Persistent gap	A competency classified as a gap and evidenced in two or more subjects.
Isolated gap	A gap evidenced in one subject only.
Classification basis	Which rule produced a label: relative position, absolute floor, absolute ceiling, insufficient data.
Attainment	A student's weighted mean score over the observations evidencing one competency. Defined in Appendix B.3.
MAD	Median absolute deviation, unscaled, over a student's competency attainments. Defined in Appendix B.3.
Relative position	How many MADs a competency's attainment sits above or below the student's own median. Defined in Appendix B.3.
Grounding	Validating an LLM response against the input vocabulary, by exact name, before using it.
Staff gate	The pending / confirmed / rejected review that must be passed before a gap report is produced.
Rubric fill	The level a marker selected for one criterion on one submission, plus their remark. Only in the database.
Reference run	A committed clustering cache, review file and plans so the system runs without an LLM (PR #22).
LjaDataset	The in-memory dataset every loader produces and every stage consumes.
Priority group	1st: at least one competency below the absolute floor. 2nd: no floor breach but a persistent gap. 3rd: isolated gaps only. Combines existing classifications; adds no threshold (D14).
Lowest gap	A flagged student's lowest flagged attainment, with its MAD position where the relative rule decided it; the ordering key within a priority group.
Study strategy	One entry per gap saying how to study it, using techniques from a closed, evidence-based list (Lja.strategy).
Practice quiz	Questions per gap, each tied to one SILO and one assessment of the student; multiple choice or a written task with marking points (Lja.quiz, PR #52).
Educator review	The quiz's blind second pass: the model answers each question without the key and the code compares (§3.10). Shown as educator-only notes.
Pseudonym	In the export, an HMAC-SHA256 of the student id keyed on LJA_EXPORT_SALT; stable under one salt, irreversible without it.
Competency tagger	Embeddings plus k-means over a catalogue's SILOs, labelled by the LLM; produces the generator's ground truth (D18).
Subject catalogue	The YAML that drives the generator: subjects, SILOs with competency tags, assessments, programs, and (from the handbook) each subject's synopsis.
Trajectory	One student's attainment in one competency, subject by subject, in the intended order, with the subjects not yet sat ahead. Order, not time (§3.11).
Subject sequence	The declared order of subjects (LJA_SUBJECT_SEQUENCE); unlisted subjects fall back to the year digit in their code, and every trajectory says which rule ordered it.

Appendix B. Metrics reference

This appendix defines every figure the system calculates or displays: its formula, its inputs, where it is calculated, where it is shown, and the thresholds it depends on. Pass/fail checks such as clustering coverage and grounding are not metrics; they are described in §3.

Where each metric lives. Every metric in this appendix is on `main` at 2 Oct 2026 (the catalogue evaluation metrics in B.6 merged in PR #24, the outcome-quality metrics in B.5 in PR #28, the priority groups and the flagged-mark histogram in PR #43).

For the reasoning behind these rules, with flowcharts and one worked student per priority group, see the User Document, Chapters 2 to 4. This appendix is the formal definition; that document is the explanation.

Conventions. Scores and attainments are percentages from 0 to 100. Weights are fractions. “Gap” means a classification of either *persistent gap* or *isolated gap*.

Thresholds are proposals. Every threshold in B.3 is a proposed default, not a ratified value. The project owner confirmed there is no institutional figure to match, so ratifying them is a team decision (action A-01 in `docs/meetings/actions.md`). The reasoning behind each value is in `docs/adr/0001-relative-gap-detection.md` and in the comments in `python/lja/config.py`.

B.1 Summary

Metric	Unit	Calculated in	Shown on	Status
Subject total, average total, performance band	% / label	Supplied in the workbook	Dashboard student list and cohort statistics	Input
Competency attainment	% (1 dp)	<code>lja/model/gap_detection.py</code>	Gap report, student page, plans	main
Profile median and MAD	% points	<code>gap_detection.py</code> (<code>profile_spread</code>)	Used in classification	main
Relative position	MAD units (2 dp)	<code>gap_detection.py</code>	Gap report, student page	main
Gap classification and basis	label	<code>gap_detection.py</code> (<code>_classify</code>)	Gap report, student page, cohort chart	main
Subjects evidencing, observations	count	<code>gap_detection.py</code>	Gap report	main
Per-subject attainment	% (1 dp)	<code>lja/model/gap_evidence.py</code>	Student page evidence panel, plans	main
Trend across subjects	label	<code>trajectory.py</code> via <code>gap_evidence.describe_trend</code>	Student page, plans, export, outcome-quality progressions	IOLG-106 branch
Trajectory: points in sequence, delta, basis, subjects ahead	list, % points, label	<code>trajectory.py</code>	Gap card subject chain, <code>/run</code>	IOLG-106 branch
Future subjects sharing a competency	list	<code>gap_evidence.py</code>	Student page, plans	main
Gap and strength counts	count	<code>lja/dashboard/app.py</code>	Student list, student page	main
Cohort statistics and histogram	% (2 dp), count	<code>lja/dashboard/stats.py</code>	Dashboard home and cohort pages	main
Priority group, lowest gap, below-floor count	label, %, count	<code>lja/dashboard/app.py</code>	Students page, cohort pages, <code>/run</code>	main
Flagged-mark histogram	count per 5-point bin	<code>app.py</code>	<code>/run</code>	main
Silo and subject quality, gap rate, health	%, count	<code>lja/model/silo_quality.py</code>	<code>/silos</code> page and its lists	main
Competency progression and delta	% points	<code>silo_quality.py</code>	<code>/competency/<slug></code> page	main
Cohort gap rate and proficient rate per competency	%	<code>lja/export.py</code>	<code>cohort.csv</code>	main
Planted-gap recall, unplanted flags, clustering precision and recall	%, count	<code>lja/data/catalogue_verify.py</code>	Command-line output	main

B.2 Figures supplied in the workbook

These come from the owner’s workbook, or from the generator when it writes a workbook in the same shape. LJA reads them but does not recalculate them.

Figure	Definition	Worked example (STU0001, CSE100F)
Score	The mark for one assessment, 1 to 100	Test: 54
Weight	The assessment’s share of the subject total. A subject’s weights sum to 1.	Test: 0.15
Weighted score	$\text{score} \times \text{weight}$	$54 \times 0.15 = 8.1$

Figure	Definition	Worked example (STU0001, CSE100F)
Subject total	Sum of the subject's weighted scores, which is the weight-weighted mean score	$8.1 + 9.8 + 12.75 + 20.0 = 50.65$
Average total	The unweighted mean of the student's subject totals. Credit points are not used.	Mean of 50.65, 50.80 and 48.40: 49.95
Performance band	A label per student. The supplied data fits boundaries at 50, 60, 70 and 80: <i>At risk</i> below 50, <i>P range</i> 50 to under 60, <i>C range</i> 60 to under 70, <i>D range</i> 70 to under 80, <i>HD/D range</i> 80 and above.	49.95: <i>At risk</i>

The band boundaries are **inferred from the supplied data**, not documented by the owner; the generator reproduces them. LJA does not use the band in any classification. The workbook's *At risk* band is not the dashboard's at-risk cohort, which is deliberately undefined (B.4).

B.3 Competency attainment and gap classification

Calculated by `compute_gaps()` in `lja/model/gap_detection.py` and written to `gap_report.csv`, one row per student and competency.

Observation. One observation is one (student, assessment, SILO) combination where the SILO belongs to the competency. An assessment that covers k of the competency's SILOs contributes k observations, each carrying the assessment's full score and weight. This is a documented approximation: the workbook has one score per assessment, not one per SILO.

Competency attainment. For student s and competency c , over all observations i :

$$A_{s,c} = \frac{\sum_i \text{score}_i \times \text{weight}_i}{\sum_i \text{weight}_i}$$

Reported to one decimal place. *Worked example:* STU0003 in Data Structures and Algorithms has 15 observations with $\sum \text{score} \times \text{weight} = 305.85$ and $\sum \text{weight} = 4.80$, so $A = 63.7\%$. The full table is in `docs/handover/learning-plan-traceability.md`.

Subjects evidencing is the number of distinct subjects among the observations. **Observations** (`n_observations`) is their count.

Profile median and median absolute deviation (MAD). A student's profile is their attainments across all their competencies. With m the median of the profile:

$$\text{MAD} = \text{median}_c |A_{s,c} - m|$$

MAD is deliberately unscaled: it is not multiplied by 1.4826 to approximate a standard deviation, because the cutoffs are expressed directly in MAD units. Median and MAD are used instead of mean and standard deviation because a student has only four to eight competencies, and one very low result would drag a mean far enough to hide the rest.

Relative position.

$$r_{s,c} = \frac{A_{s,c} - m}{\text{MAD}}$$

Shown to two decimal places; the unrounded value is used for comparisons. It is recorded only when the relative rule decided the classification, and is empty otherwise.

Classification. The rules are applied in this order, and the first that applies decides:

1. **Absolute floor.** If $A < 50$, the competency is a gap. Basis: *absolute floor*.
2. **Absolute ceiling.** If $A \geq 75$, it is *proficient*. Basis: *absolute ceiling*.
3. **Too little profile to judge.** If the student has fewer than 4 competencies, or $\text{MAD} < 1.0$, it is *proficient* when $A \geq 65$ and *developing* otherwise. Basis: *insufficient data*.
4. **Relative position.** If $r \leq -1.0$, it is a gap. If $r \geq +1.0$, it is *proficient*. Otherwise it is *developing*. Basis: *relative position*.

A gap is a **persistent gap** when subjects evidencing is 2 or more, and an **isolated gap** otherwise. The guards come first because a uniformly weak student has a flat profile and would otherwise be told they have no gaps, and a uniformly strong student would have a merely very good competency flagged.

Worked example: STU0003's profile is 62.7, 63.7, 65.0, 65.1 and 66.0, so $m = 65.0$ and $\text{MAD} = 1.0$. Data Structures has $r = -1.28$, which is at or below -1.0 , so it is a gap. It is evidenced in two subjects, so it is a *persistent gap*.

Thresholds. All are environment variables read in `lja/config.py`. `python -m lja.cli` also accepts each as a flag, except the fallback proficient threshold, which is set only by its variable.

Threshold	Variable	Default	Why this value	Status (all A-01)
Absolute floor	LJA_GAP_ABSOLUTE_FLOOR	50.0	The Fail/Pass boundary	Proposal
Absolute ceiling	LJA_GAP_ABSOLUTE_CEILING	75.0	The Distinction boundary	Proposal
Relative gap cutoff	LJA_GAP_RELATIVE_GAP_CUTOFF	-1.0	One full MAD below the student's own median	Proposal
Relative strong cutoff	LJA_GAP_RELATIVE_STRONG_CUTOFF	+1.0	The mirror of the gap cutoff	Proposal

Threshold	Variable	Default	Why this value	Status (all A-01)
Minimum competencies	LJA_GAP_MIN_COMPETENCIES	4	Below four, any spread statistic is noise; supplied students carry four to eight	Proposal
Minimum spread	LJA_GAP_MIN_SPREAD	1.0	Attainment is reported to one decimal place, so a MAD under one point is flat within the figures' own resolution. On the supplied data this sends 48% of students down the relative rule; the first draft's 2.0 sent 10%.	Proposal
Fallback proficient	LJA_GAP_FALLBACK_PROFICIENT	65.0	The previous absolute threshold, kept so the fallback reproduces earlier behaviour	Proposal

Measured effects: on the supplied workbook, see the ADR. On the 5000-student generated cohort, see `data-fixtures/README-handbook-cohort.md`, which reports gap detection at cutoffs of -1.0 , -1.5 and -2.0 .

B.4 Per-student evidence and dashboard metrics

Per-subject attainment (`subject_breakdown()` in `gap_evidence.py`). The same weighted mean as B.3, restricted to one subject, except that **each assessment counts once** even when it covers several of the competency's SILOs. Because B.3 counts once per SILO, per-subject figures need not combine exactly into the overall attainment. *Example*: STU0003's CSE2ALG figure is 64.1% here, and would be 64.0% if counted per SILO. See B.7.

Subject order (`SubjectSequence`, IOLG-106). A subject listed in `LJA_SUBJECT_SEQUENCE` takes its place in that list (*declared sequence*). Otherwise its **year level** is the first digit after the leading letters of its code, CSE2ALG is year 2, and it sorts after every declared subject, by year (*year digit*). A code with neither has no order (*unknown*), is listed last and takes no part in a trend. Nothing is guessed.

Trend across subjects (`describe_trend()`, now computed by `trajectory.py`). Take the per-subject attainments of the ordered subjects the student has sat, in sequence order. If there are fewer than two, the trend is *insufficient evidence*. Otherwise, with Δ the last subject's attainment minus the first's, rounded to one decimal place:

Condition	Trend
$\Delta > +5$	improving
$\Delta < -5$	declining
otherwise	stable

The band is `LJA_TREND_STABLE_BAND`, default 5 points, shown on `/run`; a reasoned starting point, not a measured threshold, and not ratified. A trajectory's **basis** records which rule ordered the subjects sat: *declared sequence*, *year digit*, *mixed*, or *none*.

Subjects ahead (formerly *future subjects sharing a competency*). Subjects in the competency's cluster that the student has not sat, in sequence order, so the first is where the competency is assessed next. A subject the student has sat with no result touching the competency is *taken*, not ahead. These are places to prepare, never subjects to avoid.

Counts on the student list and student page.

Count	Definition
Persistent gaps	The student's competencies classified <i>persistent gap</i>
Isolated gaps	The student's competencies classified <i>isolated gap</i>
Strengths	The student's competencies classified <i>proficient</i> , on any basis. Listed furthest above the student's median first, then by attainment.

Highlighted students. On the student list, a student's link is shown in the persistent-gap colour when they have at least one persistent gap. The underlying style is named `at-risk` for historical reasons; it is not the at-risk cohort below.

Cohorts. *All students*, *Students with a persistent gap* (at least one competency classified *persistent gap*), and the three priority groups below. An *at-risk* cohort is deliberately not defined, because its threshold is a team decision (action A-01); a test asserts it is absent.

Priority groups (`app.py`, IOLG-134). A student is *flagged* when any competency is classified a gap. Flagged students are placed in exactly one group by rules the classifier already applied, with no new threshold:

Group	Rule
1st	At least one competency classified on the <i>absolute floor</i> basis (attainment below 50)
2nd	Not 1st, and at least one <i>persistent gap</i>
3rd	Not 1st or 2nd: <i>isolated gaps</i> only

Lowest gap is the student's lowest attainment among their gap competencies; where the relative rule decided it, its position in MAD units is shown beside it. Within a group, students are ordered by lowest gap ascending. **Below floor** is the count of a student's competencies on the absolute-floor basis. The `/run` histogram bins every flagged competency's attainment in 5-point steps from 0 to 100 and colours the bins under the floor.

Cohort statistics (`summarise()` in `lja/dashboard/stats.py`), over the students' average totals:

Statistic	Definition
n	Number of students in the cohort
Mean, median	Arithmetic mean and median
Variance, standard deviation	Population values (divide by n , not $n - 1$): the cohort is every student held, not a sample
Minimum, maximum	Smallest and largest
Q1, Q3, IQR	Quartiles by the inclusive method; IQR = Q3 - Q1

All are rounded to two decimal places. A statistic that is undefined, such as the quartiles of one student, is shown as a dash, never as zero.

Distribution histogram (`histogram()`). Fixed 10-point bins from 0 to 100, so two cohorts can be compared on the same axis. Each bin includes its lower bound and excludes its upper, except the top bin, which includes 100. Values outside 0 to 100 are placed in the end bins, not dropped.

Classification counts (cohort chart). The number of (student, competency) rows in each classification, for the students in the cohort.

B.5 Outcome-quality metrics

Calculated in `lja/model/silo_quality.py` and shown on the `/silos` page, its filtered lists (`/silos/list/<all|flagged|orphan|unassessed|vague>`), and the `/competency/<slug>` pages.

Per SILO.

Metric	Definition
Mean attainment	Weighted mean of score by weight over every result row, for all students, whose assessment covers the SILO. If the weights sum to zero, the unweighted mean. Two decimal places.
Students evidenced	Distinct students with at least one such row
Assessments	Number of Assessment Map entries that list the SILO
Cluster span	Distinct subjects in the SILO's competency cluster
Flagged	The clustering flagged the SILO, with a reason
No cross-subject link	Cluster span of 1
Not assessed	Assessments = 0
Vague wording	The SILO text contains a term classed as vague (below)
Gap rate	Of the students evidenced, the percentage whose overall classification in the SILO's competency is a gap

The SILO gap rate uses each student's overall verdict for the competency, not a verdict about this SILO alone.

Vocabulary classes. Each distinct content word of a SILO is matched against two stem lists by prefix, measurable first:

- **Measurable:** names something a student does that can be observed and marked, for example *analyse, apply, design, implement, evaluate, compare, construct, explain, identify, solve, justify*.
- **Vague:** names a mental state that cannot be observed, for example *understand, appreciate, be aware, be familiar, know, learn, gain, recognise, comprehend, grasp*.
- **Other:** everything else.

Qualifiers such as *basic* or *general* are deliberately not classed as vague. The classes follow the assessability test of Bloom (1956) and Anderson and Krathwohl (2001). The two lists are the team's judgement, are in the code as `VAGUE_STEMS` and `MEASURABLE_STEMS`, and have not been ratified.

Per subject.

Metric	Definition
Health	Percentage of the subject's SILOs with none of the four issues (flagged, no cross-subject link, not assessed, vague wording). Deliberately an unweighted count; the four issue counts are shown beside it.
Mean attainment	Weighted mean of score by weight over every result row in the subject
Gap rate	Over every (student in the subject, competency taught by the subject) pair, the percentage classified as a gap

Vocabulary. A term's count is the number of SILOs containing it, counted once per SILO. Its mean attainment is the mean of those SILOs' mean attainments.

Competency progression, for each competency taught in two or more subjects, subjects in year order:

- **Per-subject mean attainment:** the weighted mean over rows covering the competency's SILOs in that subject.
- **Per-subject gap rate:** the percentage of students evidenced in that subject whose overall classification in the competency is a gap.
- **Delta:** the last measured subject's mean attainment minus the first's.
- **Trend:** the same ± 5 -point band as B.4.

B.6 Evaluation metrics for generated cohorts

Calculated by `python -m lja.data.catalogue_verify`, which compares a gap report against the answer key the generator writes (`<workbook>.truth.json`). These measure the system, not a student. The generator's own

parameters are described in `data-fixtures/README-handbook-cohort.md`. The dashboard never reads the truth file's answer key; `/run` reads only its seed and parameters.

Metric	Definition
Planted-gap recall (any)	Of the students given a deliberately weakened competency, the percentage the gap report classifies as a gap in that competency
Planted-gap recall (persistent)	The same, counting only <i>persistent gap</i>
Unplanted persistent flags	Students with no planted gap who have at least one <i>persistent gap</i>
Consistent with true ability	Of those, the number whose flagged competency is among their weakest third by true ability: the lowest $\max(1, \lfloor n/3 \rfloor)$ of their n generated abilities
Clustering pair precision, recall, F1	Over every pair of SILOs: a true positive is a pair grouped together by both the LLM and the answer key; a false positive is grouped only by the LLM; a false negative only by the key. $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$

Unplanted flags are not automatically false positives: every generated student has real variation between competencies, so a flag in their genuinely weakest competency is a correct detection of a gap nobody planted.

B.7 Known inconsistencies and open decisions

Item	Detail	Needed
Two ways of counting observations	Overall attainment (B.3) counts once per SILO; per-subject attainment (B.4) counts once per assessment. Displayed per-subject figures can differ slightly from what would combine into the overall figure.	Choose one rule and apply it in both functions
One score, several SILOs	An assessment's single score counts as full evidence for every SILO it covers (B.3)	Confirm with the owner whether per-SILO marks will exist
Gap thresholds	Every value in the B.3 threshold table	Team ratification (action A-01)
Trend band	LJA_TREND_STABLE_BAND, ± 5 points, used in B.4 and B.5	Ratify, or tune on a real multi-year dataset
Subject order	The declared sequence is the three supplied subjects; every other subject orders by its year digit	The owner's course map (FR-1.7)
Vague and measurable lists	The stem lists in B.5	Review by teaching staff
Performance band boundaries	Inferred from the supplied data (B.2)	Confirm with the owner
Gap rate meaning	SILO and subject gap rates use each student's overall competency verdict (B.5)	Keep, or define a per-subject verdict
At-risk cohort	Deliberately undefined (B.4)	Team decision (action A-01)
"At risk" naming	The workbook's <i>At risk</i> band, the student-list <i>at - risk</i> highlight and the undefined at-risk cohort are three different things (B.2, B.4)	Rename the highlight style once the cohort is defined

Appendix C. Source documents

Document	Location	Use
Tender	Tender Document.docx (23 Aug 2026)	Requirements R1–R12, evaluation matrix, risk register, budget
Owner requirements from the 11 Aug call	docs/LJA -- Requirements_for_SILO_analysis.pdf	FR/NFR list
ADR 0001 relative gap detection	docs/adr/0001-relative-gap-detection.md	Thresholds, rejected alternatives, flat-profile measurement
ADR 0002 no-embedding clustering	docs/adr/0002-no-embedding-silo-clustering.md	Tender deviation, review triggers
Sprint plan Rev 5 (binding) and Rev 6	docs/LJA_Sprint_Plan_3-6_rev5.pdf, sprint5-plan-rev6-2026-09-23.md	Definition of Done, sprint dates
Actions register	docs/meetings/actions.md	Every A-nn cited above
Sprint 4 report and Sprint 5 plan	sprint4-report-sprint5-plan-2026-09-23.md	Verified points, handover requirements
Sprint 5 briefs	sprint5-briefs/	UAT rows, security evidence steps
Validation worksheet	docs/validation-worksheet.md	IOLG-110: seven profiles worked by hand
Grounding audit	docs/grounding-audit.md, docs/sprints/sprint-5/grounding-audit/	IOLG-121: ten plans, machine results, assisted pre-review, evidence pairs
Security evidence	docs/security-evidence.md, docs/sprints/sprint-5/security-evidence/	IOLG-111: the six checks and the scanner log
Compliance checklist	docs/compliance-checklist.md	IOLG-117: 40 tender items with evidence
Rebuild check	docs/rebuild-check.md, docs/sprints/sprint-5/rebuild/	IOLG-118: why the clean rebuild has not run, and preflight findings
UAT checklist and findings	docs/uat-checklist.md, docs/sprints/sprint-5/uat/	IOLG-130, IOLG-129: S1–S6 preliminary results, UAT-01 and UAT-02
Risk register	docs/risk-register.md	IOLG-125: twelve rows re-scored
Cluster review	docs/cluster-review-sprint5.md, data-fixtures/reference-run/silo_clustering.review.sui-lung.json	IOLG-124/133: staff verdicts on the five clusters
Export schema and sample	docs/export-schema.md, docs/export-sample.ipynb	IOLG-120: every column of the export, a notebook that reads one
Large cohort	data-fixtures/README-handbook-cohort.md	IOLG-113: how the 100-subject, 5,000-student cohort is built and why
Bundle READMEs	README.md, python/README.md, sql/README.md, devenv/README.md, data-fixtures/README.md	Detail behind §4 and §5
Lecture summaries	docs/lecture_summaries/	Course guidance on handover format (W1–W10)

Appendix D. Change log

Version	Date	Author	Change
0.1	26 Sep 2026	Allan Campton (drafted with Claude Code)	First draft from the codebase and documentation at <code>main ac75a8e</code> plus open PRs #22–#25
0.2	27 Sep 2026	Allan Campton (drafted with Claude Code)	Added Appendix B, Metrics reference; former Appendices B and C are now C and D. Added Figure 5b and the pointer to the learning-plan traceability document.
0.3	29 Sep 2026	Allan Campton (drafted with Claude Code)	Companion-document row and Appendix B now point at the User Document, which holds the explanation and worked examples behind the classification rules.
0.4	2 Oct 2026	Allan Campton (drafted with Claude Code)	Brought up to <code>main</code> at <code>cd98ce8</code> (24 pull requests merged since 0.1's baseline) plus open PRs #25 and #52. New §3.10 on the generated-artefact family and the practice quiz; decisions D14–D18; threat model, limitations, configuration, commands, repository map, troubleshooting, debt register and test tables updated; §6.5 now records the Sprint 5 validation results and what is still open; §7.3 pre-filled with the team's preliminary UAT results; §8 and Appendix B reconciled with what is merged; Appendix C lists the Sprint 5 evidence files. The §3 diagrams were not re-rendered (debt item 22).
0.5	2 Oct 2026	Allan Campton (drafted with Claude Code)	IOLG-106 trajectory model: new §3.11 with Figure 5c and the unfiltered-ahead finding, decision D19, story D12, configuration rows, repository map, debt items 7 and 8 resolved to configuration, §8.2 and Appendix B reconciled. Ships on the IOLG-106 branch.
0.6	2 Oct 2026	Allan Campton (drafted with Claude Code)	IOLG-137: Generate and Regenerate buttons on the student page, opt-in. Decision D20, threat-model row, configuration and command rows, repository map, §5.5 recipe, troubleshooting rows, test row D13. IOLG-106 marked merged (PR #53). Ships on the IOLG-137 branch.